

ABSTRACT

The E-Dairy Farm System is a comprehensive, online-based platform designed to streamline dairy farm operations by integrating key activities such as cattle management, milk production, product sales, and online shopping for dairy products. Traditional farming methods involve manual record-keeping, which can lead to inefficiencies and errors. This system automates crucial tasks like tracking milk production, monitoring cattle health, managing inventory, and maintaining daily transactions. It has three primary users: Administrator, Staff, and Public. The Administrator oversees all farm operations, including product management, cattle health records, and user permissions. The Staff is responsible for updating details on cattle, milk production, and farm activities. The Public (customers) can browse and purchase dairy products online, making transactions easier and more convenient. Additionally, the system ensures secure access for different users, preventing unauthorized modifications and enhancing data reliability.

With features like advanced search options, automated transactions, and an intuitive user interface, the system reduces manual workload, enhances efficiency, and improves data accuracy. It enables real-time tracking of production and inventory, helping farmers make informed decisions. Additionally, it serves as a platform for online ordering and sales, allowing dairy farmers to reach a broader market while streamlining transactions with secure payment options. The system also enhances cattle health management by providing automated monitoring, reducing disease risks, and improving livestock welfare. By integrating digital solutions into traditional dairy farming, the E-Dairy Farm System modernizes farm operations, minimizes errors, increases profitability, and ensures a steady supply of high-quality dairy products. It also improves customer experience by allowing them to track orders, view product availability, and receive timely updates. This technology-driven approach makes dairy farming more efficient, transparent, and adaptable to evolving market demands, ultimately contributing to the industry's sustainable growth.

CHAPTER 1

INTRODUCTION

1.1 ABOUT THE PROJECT

The E-Dairy Farm System is a modern digital platform designed to streamline and enhance dairy farm management. It automates key processes such as cattle monitoring, milk production tracking, inventory management, and online sales of dairy products. By reducing manual errors and improving efficiency, the system ensures accurate record-keeping and smooth daily operations. It serves three primary users: Administrator, Staff, and Public, each playing a crucial role in farm management and product distribution. The Administrator oversees farm operations, manages inventory, and handles user access, while the Staff updates cattle and milk production details. The Public, or customers, can browse and purchase dairy products online, making transactions more convenient. Additionally, the system offers real-time data access, secure payment options, and automated health monitoring for livestock, reducing disease risks and improving productivity. By integrating digital solutions, the E-Dairy Farm System modernizes traditional dairy farming, making it more efficient, profitable, and sustainable.

MODULE DESCRIPTION

The **E-Dairy Farm System** is divided into several key modules, each designed to handle a specific aspect of dairy farm operations. Below is a detailed breakdown of each module and its functionality.

- ADMINISTRATOR MODULE
- STAFF MODULE
- USER MODULE (CUSTOMER)
- SHOPPING MODULE
- INVENTORY MANAGEMENT MODULE

1.1.1 ADMINISTRATOR

The Administrator has full access and control over the entire system. This module is designed for high-level management and oversight.

- **Login/Logout:** Secure authentication for admin access.
- **Manage Users:** Add, update, or remove staff members. Set access levels for different roles.

- **Farm Management:** Oversee all activities, including cattle management, milk production, and product sales.
- **Milk Product Management:** Add and update details of dairy products available for sale.
- **Report Generation:** Generate reports for milk production, sales, inventory, and cattle health.
- **Health Monitoring:** Track and manage the health status of cattle, including vaccination schedules, treatment records, and performance.
- **Inventory Management:** Monitor stock levels for feed, equipment, and dairy products.

1.1.2 STAFF

Staff members are responsible for day-to-day operations on the farm, including managing cattle, milk production, and maintaining the inventory. They have limited access compared to the Administrator.

Login/Logout: Secure access for staff with appropriate permissions.

Cattle Data Entry & Updates:

- Add new cattle to the system, including breed, age, health status, and milk production history.
- Update health information regularly (vaccinations, breeding details, milk yield).
- Maintain detailed logs of each cow's progress and performance.

Milk Production Recording:

- Log daily milk production for each cow or group of cows.
- Monitor and update the quantity of milk produced on a daily basis.
- Track fluctuations in milk yield and alert the Administrator in case of unusual changes.

Inventory Updates:

- Manage day-to-day updates to the dairy product inventory.
- Mark products as sold or out of stock after customer purchases.
- Monitor and manage product expiration dates to ensure timely sales.

Sales Support:

- Assist in fulfilling customer orders if needed.
- Handle packaging and preparation for shipment.
- Keep track of inventory as products are sold and ensure products are correctly labeled.

Routine Tasks:

- Regularly monitor and report cattle health.
- Ensure that farm equipment and facilities are maintained and operational.
- Assist in quality control for dairy products.

1.1.3 USER (CUSTOMER)

The Public module is designed for external users (customers) who are interested in purchasing dairy products. It provides them with an easy-to-use platform for browsing products and placing orders online.

- **Login/Logout:** Secure access for user with appropriate permissions.
- **Browse Products:** View available milk and dairy products listed for sale.
- **Online Ordering:** Purchase dairy products through the platform's online store.
- **Order History:** Track past orders and delivery status.
- **Payment Gateway:** Process payments for online orders securely.
- **Product Reviews:** Rate and review purchased products.
- **Search and Filter Options:** Advanced search functionality to filter products based on categories, availability, and other parameters.

1.1.4 SHOPPING

This module provides a seamless e-commerce experience for customers to browse and purchase products directly from the farm.

- **Product Listings:** Display a comprehensive list of available dairy products, including milk, cheese, butter, yogurt, and other processed dairy items.
- **Product Categories:** Products can be grouped into categories (e.g., Milk, Cheese, Butter, Yogurt, etc.) to make browsing easier for customers.
- **Product Details:** Each product page provides detailed information, such as product description, price, weight/quantity, nutritional information, and any special features (e.g., organic, lactose-free).
- **Search and Filters:** Customers can search for specific products by name or filter results based on categories, price range, brand, or other parameters.

- **Image Gallery:** Each product listing includes high-quality images, allowing customers to view the product before purchasing.
- **Add to Cart:** Customers can add items to their shopping cart with a single click. The cart displays the list of selected items along with their quantities and prices.
- **Remove Items:** Customers can remove any items they no longer wish to purchase from their shopping cart.
- **Cart Summary:** The cart provides a real-time summary of the total amount due, including product costs, taxes, and any applicable shipping charges or discounts.

1.1.5 INVENTORY MANAGEMENT

Role: This module ensures the proper tracking and management of dairy products, raw materials, and supplies.

- **Product Tracking:**
 - Monitor the quantity of each dairy product in stock.
 - Maintain stock level records and alert for reordering when stock reaches a minimum threshold.
- **Expiry Date Management:**
 - Track expiration dates of perishable goods.
 - Send automated reminders for products nearing expiration.
- **Restocking and Supply Management:**
 - Generate purchase orders when stock levels fall below a predefined threshold.
 - Keep track of suppliers and manage procurement processes.
- **Stock Movement:**
 - Log the movement of stock (e.g., products added to inventory, items sold, items removed).
 - Update inventory in real-time as sales are made or new stock is received.

1.2 COMPANY PROFILE

YUVATECH Solutions Private Limited is an IT Solution provider, specialized in the development of software and web application, aiming to deliver quality services to customers. The wide stream of products and projects that we are providing is unique in the outlook of our intention. Our unique customer insights and commitment helps the way to establishing a long term support with the clientele. Our solutions are based on cutting edge, state of the art technology, and industry standard tools and proven frameworks. We take hands on approach, working with you, throughout the development process starting from initial planning and analysis, through design, development, testing, training and support. We deliver great business value to our customers and strive to achieve customer satisfaction. We look forward to build a long lasting relationship with our customers and take pride in having an impact on their success.

Mission

Our mission is to create value and make a difference in the field of education and also to provide sustainable advanced technology solutions and services to our clients. We strive to become the preferred partner of choice for both national and international clients. We aspire to be an employer of choice, creating an engaging workforce of highly skilled professionals.

Vision

Our Vision is to become leader in the field of IT Training and development. We will achieve our vision through our teamwork, innovative approach, delivering standard products and services

Our Team

YUVATECH is a young, nimble team with passion for delivering innovative software solutions, products and customer service. Our biggest strength is our team. We feature a highly experienced team of managers and diverse creative and technical teams who strongly believe in providing the upmost quality to all our clients. Our teams provide a one-stop source for all of your technology needs from IT strategy, system architecture, design, implementation, integration and support. Sharing the values with the team and providing support to individuals to solve any challenges presented to them. To meet our vision and

mission, our team regularly upgrade their creative minds with latest technology and methodologies to enhance their individual talent and knowledge.

Our Services

Being a premier web developing company, we at YUVATECH follow a strict principle to provide our clients Up-to-date services, keeping in a pace with the latest trend in the industry. We have a complete solution that ranges from creating custom website design services, web application development and in custom software development. With our custom software and web development solution, you can leverage the leading technologies and mark a sustainable place in the internet market place by gaining the competitive advantage.

At YUVATECH, we strive to give all our customers a service keeping in mind about their times, cost benefits, comfort, budget target, schedule, security and confidential issues. We offer our solutions keeping in mind about the current technology platforms of our clients and their future plans.

Our clients vary from a small enterprise to a large scale organization and corporate companies. Also, the services are offered as per clients specifications. The systematic execution of the services are done with the help of our diligent team of professionals.

Our services are

- Domain Name Registration
- Web Hosting
- Web Server Migration
- Web Design and Development
- Website Maintenance
- E-Commerce Solution
- IT Training Program

CHAPTER 2

SYSTEM ANALYSIS

System analysis works with users to identify goals and build systems to achieve them. System analysis is the important phase of any system development process. The system is studied to the minutest details and analyzed. The system analyst plays the role of an interrogator and dwells deep into the working of the present system. In analysis, a detailed study of these operations performed by a system and their relationship within and outside of the system is done. A key question considered here is “what must be done to solve the problem?”

The system is viewed as a whole and the input to the system are identified. The output from the system are traced through the various data are collected on available files, decision points and transaction handled by present system.

After all, this proposal becomes a formal agreement that paves way for the design implementation.

2.1 REQUIREMENT ANALYSIS

Requirement analysis is the process of analyzing the requirements with the potential goal of improving or modifying the existing system and its features. Analyzing the advantages and disadvantages of the current system, the proposed system can be designed which can avoid all the complexities, inabilities and the disadvantages of the existing system. The new system requirements are defined during this phase. The requirements of the desired software product are extracted.

To design a new system we need the requirements of the system. Based on the business scenario the software Requirement Specification document is prepared in this phase. The specifications are intended to guide the group through the development process. It explains all the process, activities, relationships and all other organizational objectives. With the analysis of the current system the analyst can understand the reason for automation and what effects the automation system might have.

User’s requirements have been identified as follows:

- **Pre-defined Questions**

It allows analysts to collect information about the various aspects of the system from large number of persons. The use of standardized question format can yield more reliable data than other technique.

- **Interview**

Analysts use interviews to collect information from individuals or from groups. The respondents are generally current users of the existing system.

- **Record Interview**

In record review, analysts examine information that has been recorded about the system and the users. Record inspection can be performed at the beginning of the study as an introduction and renew later in the study as a basis for comparing actual operations with what the records indicate.

PROJECT PLANNING

Even with small projects you need to know what the project aims to do, how you'll know if you've finished (and been successful, what the scope of the project will be, what resources it will be able to use, and who will be involved, but that's just the beginning.

Start a mind map if you haven't already. Think about what the project aims to produce, what it covers, what you will need to finish it and then draft the first ideas for a timetable:

Scope may be the trickiest. It should mention what's included, but it's a good idea to try to write what is **not included** as well. No, not trivial exclusions ("this project will exclude an attempt to set up a moon-base"). What's hard is to catch as many as you can of the exclusions that other player in the project might expect to be **included**.

You may not have decided much of what you write about aims, scope and resources yourself -- maybe your boss told you -- but whether you define them and then go seeking agreement, or they are handed down, make sure they have been thought through, written down and circulated. Either way if you are managing the project, the next part is down to you. And this is where the mindmap gets really useful.

Mind mapping encourages structured capture of unstructured thoughts. You could do some of this with a checklist, but a mind map lets you move quickly, skip from one area to another when you foresee consequences there, and remain open and unconstrained when planning. There will be a time for formal and detailed planning, but this is not it -- you need a broad view and the mind map approach gives that.

Next, develop the mind map to a more detailed level. You have aims, now you need a statement of requirements, and an informal agreement with the sponsors of the project, or more likely a formal agreement, and even a legal contract in some cases. Depending on

how formal your sponsors are, there may be penalties to agree at this stage and you will need to consider contingencies in cases of failure to obtain all the resources or meet certain requirements -- delivery dates especially.

The problem with linear planning is clear, now. Here the description, as is the nature of prose, is linear, but at this point in the project planning, there will actually be movement on many fronts -- timetable planning; consideration of constraints; negotiation about completion dates and penalties; factoring in the need for approvals and whether these will be available in a timely manner; and considering internal controls imposed by, say, quality processes or regulatory requirements, if these apply.

Such multiple and parallel activities are better supported by a mind map than a linear outline, a checklist or a purely text-based planning document.

This mind map continues to grow though the planning process as more and more detail is filled in, and detailed budgeting, formal timetable preparations and resource allocation get under way.

Taking regular snapshots of the mind map and keeping these intermediate stages is a good idea. It can help with future projects. Once you have found the right form of project-planning mind map for your work, you will be able to use its general structure for project after project, and tune it as your projects grow.

2.2 EXISTING SYSTEM

In the existing system, dairy farm operations are often managed manually or with basic software solutions. Key challenges of this system include:

- **Cattle Details:** Farm staff typically track cattle health, breeding, and milk production manually, often in physical ledgers or spreadsheets.
- **Milk Production & Sales:** Milk production data and sales transactions are logged manually, which can lead to human error and inefficiencies.
- **Inventory Management:** Stock of milk and other dairy products are tracked manually, making it harder to keep up with demand and supply.

2.3 PROPOSED SYSTEM

The proposed system is a modern, automated platform designed to optimize the operations of dairy farms and provide better customer service through online sales. The E-Dairy Farm System is a comprehensive solution for modernizing dairy farm operations. By automating daily tasks, offering an online sales platform, and providing real-time data analytics, it

enhances farm efficiency, improves customer satisfaction, and supports better decision-making. This system addresses the challenges faced by traditional dairy farms and provides a sustainable, scalable solution for the future of dairy farming.

- **Cattle Management:** Tracks cattle health, breeding, and milk yield automatically, reducing manual input and minimizing errors.
- **Milk Production Tracking:** Real-time data on milk production ensures accurate and up-to-date reporting.
- **Inventory Management:** Automatically tracks dairy product stocks (milk, cheese, etc.), helping to maintain balanced inventory levels.
- **Order Management:** Customers place orders, which are processed and tracked efficiently within the system, providing a seamless shopping experience.

ADVANTAGES OF PROPOSED SYSTEM :

- **Efficiency:** Automating routine tasks reduces manual workload, saving time and reducing errors.
- **Accuracy:** Real-time tracking of production, sales, and inventory improves the accuracy of farm data and business decisions.
- **Customer Satisfaction:** Providing an online shopping option for dairy products meets customer demand for convenience, enhancing the customer experience.
- **Cost Savings:** Reduced reliance on manual processes can lower operational costs, leading to increased profitability.
- **Scalability:** The system allows for easier expansion and adaptability, helping the farm grow without the need for significant manual adjustments.

PROJECT SCHEDULING

Scheduling of the software project can be correlated to prioritizing various tasks with respect to their cost, time and duration. Scheduling can be done with resource constraint or time constraint in mind. Depending upon the project scheduling methods can be static or dynamic in implementation.

2.4 FEASIBILITY STUDY

It is an analysis of the viability of an idea. It is the second major step in the system development life cycle. It focuses on helping answer the essential question of “should we proceed with the proposed project idea?”

The feasibility study is an evaluation and analysis of the potential of a proposed project which is based on extensive investigation and research to support the process of decision making. A feasibility study evaluates the project's potential for success.

Different feasibility tests are :

1. Economic Feasibility
2. Operational Feasibility
3. Technical Feasibility

ECONOMIC FEASIBILITY

Economic analysis is also known as cost/benefit analysis. It is the most frequently used method for evaluating the effectiveness of a new system. In economic analysis, the procedure is to determine the benefits and savings that are expected from a candidate system and compare them with costs.

The proposed system is economically feasible due o the following reasons :

- it does not require any additional hardware or software,
- the development cost is very less
- the cost of implementation is also very less

TECHNICAL FEASIBILITY

It considers the technical requirements of the proposed project. The technical requirements are then compared to the technical capability of the organization. The systems project is considered technically feasible if the internal technical capability is sufficient to support the project requirements.

Our proposed system is technically feasible for the following reasons :

- The software and hardware requirements for the development of this project are already available,
- The available equipments have the capacity to hold the data required to use the new system
- This system can be upgraded after development
- The proposed system can provide adequate responses to the user's inquiry.

OPERATIONAL FEASIBILITY

Proposed project would be beneficial only if they can be turned into information system that'll meet the organization operating requirements. Once the main problems faced during the development of a new system is getting acceptance from user. The test of operational feasibility asks if the system will work when developed and installed.

Our proposed system is operationally feasible due to the following reasons:

- The tools selected for developing the software are PHP, MySQL, Windows 7 or higher which are all easily available,
- It can reduce different difficulties in the existing system.

2.5 HARDWARE AND SOFTWARE SPECIFICATION

Hardware Requirements:

- Processor : Intel Pentium4
- RAM : 1GB
- Hard Disk : 250GB
- Drives : CD ROM Drive
- Display Size : 14" Color Monitor
- Screen Resolution : 1024× 768 pixels
- Color Palette : True Color (24 bit)
- Keyboard : PC/AT Enhanced Type
- Mouse : Logitech PS/2 Port Mouse
- Modem : 56Kbps

Software Requirements

- Operating System : Windows 10 or higher
- Framework : Microsoft .Net Framework 7.0
- Environment : Visual Studio 2022
- Front end : HTML,CSS,ASP.Net and C#.Net
- Back end : Microsoft Sql Server 2019 or higher
- Documentation : Microsoft Word 2016 or higher

2.5.1 SOFTWARE REQUIREMENTS

In this section a brief introduction of the technologies of the software used in the proposed system are discussed.

Visual Studio 2022

Microsoft's Visual Studio is one of the most popular developer IDEs in the world. Not only is Visual Studio modern, feature-rich, and advanced, but it also gets more frequent updates than any other IDE out there. The next version of Visual Studio is going to be Visual Studio 2022. The Visual Studio team just released the first public preview of Visual Studio 2022.

Visual Studio 2022 is much faster and more lightweight compared to the current version. The Visual Studio team has focused on making IDE more user-friendly by enhancing the overall user experience. It looks cleaner.

Visual Studio 2022 also has intelligent features that include recommendations around code cleanliness, quality, standards, and best practices.

Git Tooling

Removed the ability to revert back to the Team Explorer Git UI, making the new Git experience the only available built-in tooling.

Removed the option to install the GitHub extension from the Visual Studio Installer.

New versions of the test platform will not be able to run Generic tests and Ordered tests.

Web Tools

The Publish summary page now has actions to start/stop remote debugging and profiling under the '...' menu on the top right corner of the 'Hosting' section

The Connected Services page now has an action to launch Storage Explorer

The "ASP.NET Core Empty" template that comes with .NET 6 is using the new 'minimal APIs' paradigm for which we have started to add support.

Some of the feature that has not implemented yet but is coming in the next releases.

- Web Live Preview
- Instrumentation profiler
- Azure Cloud Service project support
- T-SQL debugger
- Web Load Test and TestController/TestAgent
- Azure DataLake
- Coded UI Test
- DotFuscator
- Incredibuild IDE integration
- IntelliCode find and replace by example

Visual Studio is an integrated development environment (IDE) developed by Microsoft. It is used to develop computer programs including websites, web apps, web services and mobile apps. Visual Studio uses Microsoft software development platforms such as Windows API, Windows Forms, Windows Presentation Foundation (WPF), Windows Store and Microsoft Silverlight. It can produce both native code and managed code.

Visual Studio includes a code editor supporting IntelliSense (the code completion component) as well as code refactoring. The integrated debugger works as both a source-level debugger and as a machine-level debugger. Other built-in tools include a code profiler, designer for building GUI applications, web designer, class designer, and database schema designer. It accepts plug-ins that expand the functionality at almost every level including adding support for source control systems (like Subversion and Git) and adding new toolsets like editors and visual designers for domain-specific languages or toolsets for other aspects of the software development lifecycle (like the Azure DevOps client: Team Explorer).

Visual Studio supports 36 different programming languages and allows the code editor and debugger to support (to varying degrees) nearly any programming language, provided a language-specific service exists. Built-in languages include C, C++, C++/CLI, Visual Basic .NET, C#, F#, JavaScript, TypeScript, XML, XSLT, HTML, and CSS. Support for other languages such as Python, Ruby, Node.js, and M among others is available via plug-ins. Java (and J#) were supported in the past.

Architecture

Visual Studio does not support any programming language, solution or tool intrinsically; instead, it allows the plugging of functionality coded as a VSPackage. When installed, the functionality is available as a Service. The IDE provides three services: SVsSolution, which provides the ability to enumerate projects and solutions; SVsUIShell, which provides windowing and UI functionality (including tabs, toolbars, and tool windows); and SVsShell, which deals with registration of VSPackages. In addition, the IDE is also responsible for coordinating and enabling communication between services.^[10] All editors, designers, project types and other tools are implemented as VSPackages. Visual Studio uses COM to access the VSPackages. The Visual Studio SDK also includes the Managed Package Framework (MPF), which is a set of managed wrappers around the COM-interfaces that allow the Packages to be written in any CLI compliant language.^[11] However, MPF does not provide all the functionality exposed by the Visual Studio COM interfaces.^[12] The services can then be consumed for creation of other packages, which add functionality to the Visual Studio IDE.

Support for programming languages is added by using a specific VSPackage called a Language Service. A language service defines various interfaces which the VSPackage implementation can implement to add support for various functionalities. Functionalities that can be added this way include syntax coloring, statement completion, brace matching,

parameter information tooltips, member lists, and error markers for background compilation. If the interface is implemented, the functionality will be available for the language. Language services are implemented on a per-language basis. The implementations can reuse code from the parser or the compiler for the language. Language services can be implemented either in native code or managed code. For native code, either the native COM interfaces or the Babel Framework (part of Visual Studio SDK) can be used. For managed code, the MPF includes wrappers for writing managed language services.

Visual Studio does not include any source control support built in but it defines two alternative ways for source control systems to integrate with the IDE. A Source Control VSPackage can provide its own customised user interface. In contrast, a source control plugin using the MSSCCI (Microsoft Source Code Control Interface) provides a set of functions that are used to implement various source control functionality, with a standard Visual Studio user interface. MSSCCI was first used to integrate Visual SourceSafe with Visual Studio 6.0 but was later opened up via the Visual Studio SDK. Visual Studio .NET 2002 used MSSCCI 1.1, and Visual Studio .NET 2003 used MSSCCI 1.2. Visual Studio 2005, 2008, and 2010 use MSSCCI Version 1.3, which adds support for rename and delete propagation, as well as asynchronous opening.

Visual Studio supports running multiple instances of the environment (each with its own set of VSPackages). The instances use different registry hives (see MSDN's definition of the term "registry **hive**" in the sense used here) to store their configuration state and are differentiated by their AppId (Application ID). The instances are launched by an AppId-specific .exe that selects the AppId, sets the root hive, and launches the IDE. VSPackages registered for one AppId are integrated with other VSPackages for that AppId. The various product editions of Visual Studio are created using the different AppIds. The Visual Studio Express edition products are installed with their own AppIds, but the Standard, Professional, and Team Suite products share the same AppId. Consequently, one can install the Express editions side-by-side with other editions, unlike the other editions which update the same installation. The professional edition includes a superset of the VSPackages in the standard edition, and the team suite includes a superset of the VSPackages in both other editions. The AppId system is leveraged by the Visual Studio Shell in Visual Studio 2008.

Features

Code Editor

Visual Studio includes a [code editor](#) that supports [syntax highlighting](#) and [code completion](#) using [IntelliSense](#) for [variables](#), [functions](#), [methods](#), [loops](#), and [LINQ](#) queries. IntelliSense is supported for the included languages, as well as for [XML](#), [Cascading Style Sheets](#), and [JavaScript](#) when developing web sites and [web applications](#). Autocomplete suggestions appear in a [modeless list box](#) over the code editor window, in proximity of the editing [cursor](#). In Visual Studio 2008 onwards, it can be made temporarily semi-transparent to see the code obstructed by it. The code editor is used for all supported languages.

The code editor in Visual Studio also supports setting bookmarks in code for quick navigation. Other navigational aids include [collapsing code blocks](#) and [incremental search](#), in addition to normal text search and [regex](#) search. The code editor also includes a multi-item [clipboard](#) and a task list. The code editor supports code snippets, which are saved templates for repetitive code and can be inserted into code and customized for the project being worked on. A management tool for code snippets is built in as well. These tools are surfaced as floating windows which can be set to automatically hide when unused or docked to the side of the screen. The code editor in Visual Studio also supports [code refactoring](#) including parameter reordering, variable and method renaming, [interface](#) extraction, and encapsulation of class members inside properties, among others.

Debugger

Visual Studio includes a [debugger](#) that works both as a source-level debugger and as a machine-level debugger. It works with both [managed code](#) as well as [native code](#) and can be used for debugging applications written in any language supported by Visual Studio. In addition, it can also attach to running processes, monitor, and debug those processes.^[24] If source code for the running process is available, it displays the code as it is being run. If source code is not available, it can show the [disassembly](#). The Visual Studio debugger can also create [memory dumps](#) as well as load them later for debugging. Multi-threaded programs are also supported. The debugger can be configured to be launched when an application running outside the Visual Studio environment crashes.

The Visual Studio Debugger allows setting [breakpoints](#) (which allow execution to be stopped temporarily at a certain position) and watches (which monitor the values of variables as the execution progresses). Breakpoints can be conditional, meaning they get

triggered when the condition is met. Code can be [stepped over](#), i.e., run one line (of source code) at a time. It can either step into functions to debug inside it, or step over it, i.e., the execution of the function body isn't available for manual inspection. The debugger supports Edit and Continue, i.e., it allows code to be edited as it is being debugged. When debugging, if the mouse pointer hovers over any variable, its current value is displayed in a tooltip ("data tooltips"), where it can also be modified if desired. During coding, the Visual Studio debugger lets certain functions be invoked manually from the [Immediate](#) tool window. The parameters to the method are supplied at the Immediate window.

Designer[edit]

Visual Studio includes many visual designers to aid in the development of applications. These tools include:

Windows Forms Designer

The Windows Forms designer is used to build [GUI](#) applications using [Windows Forms](#). Layout can be controlled by housing the controls inside other containers or locking them to the side of the form. Controls that display data (like textbox, list box and grid view) can be [bound](#) to data sources like [databases](#) or [queries](#). Data-bound controls can be created by dragging items from the Data Sources window onto a design surface.^[29] The UI is linked with code using an [event-driven programming](#) model. The designer generates either [C#](#) or [VB.NET](#) code for the application.

WPF Designer

The WPF designer, codenamed Cider, was introduced with Visual Studio 2008. Like the Windows Forms designer it supports the drag and drop metaphor. It is used to author [user interfaces](#) targeting [Windows Presentation Foundation](#). It supports all WPF functionality including [data binding](#) and [automatic layout](#) management. It generates [XAML](#) code for the UI. The generated [XAML](#) file is compatible with [Microsoft Expression Design](#), the designer-oriented product. The XAML code is linked with code using a [code-behind](#) model.

Web designer/development

Visual Studio also includes a web-site editor and designer that allows web pages to be authored by dragging and dropping widgets. It is used for developing [ASP.NET](#) applications and supports [HTML](#), [CSS](#) and [JavaScript](#). It uses a [code-behind](#) model to link with ASP.NET code. From Visual Studio 2008 onwards, the

layout engine used by the web designer is shared with the discontinued [Expression Web](#). There is also [ASP.NET MVC](#) support for [MVC](#) technology as a separate download and [ASP.NET Dynamic Data](#) project available from Microsoft.^[32]

Class designer

The Class Designer is used to author and edit the classes (including its members and their access) using [UML](#) modeling. The Class Designer can generate [C#](#) and [VB.NET](#) code outlines for the classes and methods. It can also generate class diagrams from hand-written classes.

Data designer

The data designer can be used to graphically edit [database schemas](#), including typed tables, primary and foreign keys and constraints. It can also be used to design queries from the graphical view.

Mapping designer

From Visual Studio 2008 onwards, the mapping designer is used by [LINQ to SQL](#) to design the [mapping](#) between [database schemas](#) and the [classes](#) that encapsulate the data. The new solution from ORM approach, [ADO.NET Entity Framework](#), replaces and improves the old technology.

Other tools

Properties Editor

The Properties Editor tool is used to edit properties in a GUI pane inside Visual Studio. It lists all available properties (both read-only and those which can be set) for all objects including [classes](#), forms, web pages and other items.

Object Browser

The Object Browser is a [namespace](#) and [class library](#) browser for [Microsoft .NET](#). It can be used to browse the namespaces (which are arranged hierarchically) in [managed assemblies](#). The hierarchy may or may not reflect the organization in the file system.

Solution Explorer

In Visual Studio parlance, a solution is a set of code files and other resources that are used to build an application. The files in a solution are arranged hierarchically, which might or might not reflect the organization in the file system. The Solution Explorer is used to manage and browse the files in a solution.

Team Explorer

Team Explorer is used to integrate the capabilities of Azure DevOps (either [Azure DevOps Services](#) or [Azure DevOps Server](#)) into the IDE . In addition to version control integration it provides the ability to view and manage individual work items (including user stories, bugs, tasks and other documents). It is included as part of a Visual Studio installation and is also available as a standalone download.^{[33][34]}

Data Explorer

Data Explorer is used to manage databases on [Microsoft SQL Server](#) instances. It allows creation and alteration of database tables (either by issuing [T-SQL](#) commands or by using the Data designer). It can also be used to create [queries](#) and [stored procedures](#), with the latter in either [T-SQL](#) or in [managed code](#) via [SQL CLR](#). Debugging and [IntelliSense](#) support is available as well.

Server Explorer

The Server Explorer tool is used to manage database connections on an accessible computer. It is also used to browse running [Windows Services](#), performance counters, [Windows Event Log](#) and [message queues](#) and use them as a datasource.

Dotfuscator Community Edition

Visual Studio includes a free 'light' version of [Dotfuscator](#)

Text Generation Framework

Visual Studio includes a full text generation framework called [T4](#) which enables Visual Studio to generate text files from templates either in the IDE or via code.

ASP.NET Web Site Administration Tool

The [ASP.NET Web Site Administration Tool](#) allows for the configuration of [ASP.NET](#) websites.

Visual Studio Tools for Office

[Visual Studio Tools for Office](#) is a SDK and an add-in for Visual Studio that includes tools for developing for the [Microsoft Office](#) suite. Previously (for Visual Studio .NET 2003 and Visual Studio 2005) it was a separate SKU that supported only [Visual C#](#) and [Visual Basic](#) languages or was included in the Team Suite. With Visual Studio 2008, it is no longer a separate SKU but is included with Professional and higher editions. A separate runtime is required when deploying VSTO solutions.

Testing tools

Microsoft Visual Studio can write high-quality code with comprehensive testing tools to aid in the development of applications. These tools include:

[Unit testing](#), IntelliTest, Live Unit Testing, Test Explorer, CodeLens test indicators, Code coverage analysis, Fakes

ASP.Net

ASP.NET is an open source web framework, created by Microsoft, for building modern web apps and services with .NET. ASP.NET is cross platform and runs on Windows, Linux, macOS, and Docker. ASP.NET is a web application framework designed and developed by Microsoft. ASP.NET is open source and a subset of the **.NET Framework** and successor of the classic ASP(Active Server Pages). With version 1.0 of the .NET Framework, it was first released in January 2002. So a question comes to mind that which technology we were using before the year 2002 for developing web applications and services? Answer is *Classic ASP*. So before .NET and ASP.NET there was Classic ASP. ASP.NET is built on the **CLR(Common Language Runtime)** which allows the programmers to execute its code using any .NET language(C#, VB etc.). It is specially designed to work with HTTP and for web developers to create dynamic web pages, web applications, web sites, and web services as it provides a good integration of HTML, CSS, and JavaScript. .NET Framework is used to create a variety of applications and services like Console, Web, and Windows, etc. But ASP.NET is only used to create web applications and web services. A web application is an application installed only on the web server which is accessed by the users using a web browser like Microsoft Internet Explorer, Google Chrome, Mozilla FireFox, Apple Safari, etc. There are also some other technology like Java, PHP, Perl, Ruby on Rails, etc. which can be used to develop web applications. Web applications provide the cross-platform feature. The user needs only a web browser to access a web application. The web applications which are developed using the .NET framework or its subsets required to execute under the *Microsoft Internet Information Services(IIS)* on the server side. Don't confuse in the terms ASP.NET, ASP.NET core, ASP.NET MVC, etc. ASP(Active Server Pages) supports a lot of development models which are as follows:

- Classic ASP: It is the first server side scripting language developed by Microsoft.
- ASP.NET: It is web development framework and successor of Classic ASP. ASP.NET 4.6 is the latest version.
- ASP.NET Core: In November 2015, Microsoft released the 5.0 version of ASP.NET which get separated later and known as ASP.NET Core. Also, it is considered as an important redesign of ASP.NET with the feature of open-source and cross-platform. Before this version, ASP.NET is only considered as Windows-only version.

- ASP.NET Web Forms: These are the event-driven application model which are not considered a part of the new ASP.NET Core. These are used to provide the server-side events and controls to develop a web application.
- ASP.NET MVC: It is the Model-View-Controller application model which can be merged with the new ASP.NET Core. It is used to build dynamic websites as it provides fast development.
- ASP.NET Web Pages: These are the single page application which can be merged into ASP.NET Core.
- ASP.NET API: It is the Web Application Programming Interface(API).

Also, to create web applications ASP.NET provide the 3 development styles which are ASP.NET Web Pages, ASP.NET MVC, Web Forms.

There are a lot of reasons which makes the ASP.NET popular among the developers. Some of the reasons are listed below: Extending .NET Framework: ASP.NET is a subset of .NET Framework as it extends the .NET Framework with some libraries and tools to develop web apps. The thing that it adds to the .NET Framework is Libraries for common web patterns like MVC, Editor Extensions, the base framework to process the web requests, and web-page templating syntax like Razor, etc. Performance: It is faster than the other web frameworks available in the market. Backend Code: With the help of ASP.NET you can write the backend code for data access and any logic in C#. Dynamic Pages: In ASP.NET, Razor provides the syntax for developing the dynamic web pages with the help of C# and HTML. ASP.NET can be integrated with JS(JavaScript) and it also includes the frameworks like React and Angular for the SPA(Single Page Application.) Supporting different OS: You can develop and execute ASP.NET apps on Windows, Linux, Docker, and MacOS. The Visual Studio provides the tools to build .NET apps different OS.

ASP.NET Web Forms Model

ASP.NET web forms extend the event-driven model of interaction to the web applications. The browser submits a web form to the web server and the server returns a full markup page or HTML page in response.

All client side user activities are forwarded to the server for stateful processing. The server processes the output of the client actions and triggers the reactions.

Now, HTTP is a stateless protocol. ASP.NET framework helps in storing the information regarding the state of the application, which consists of:

- Page state

- Session state

The page state is the state of the client, i.e., the content of various input fields in the web form. The session state is the collective information obtained from various pages the user visited and worked with, i.e., the overall session state. To clear the concept, let us take an example of a shopping cart.

User adds items to a shopping cart. Items are selected from a page, say the items page, and the total collected items and price are shown on a different page, say the cart page. Only HTTP cannot keep track of all the information coming from various pages. ASP.NET session state and server side infrastructure keeps track of the information collected globally over a session.

The ASP.NET runtime carries the page state to and from the server across page requests while generating ASP.NET runtime codes, and incorporates the state of the server side components in hidden fields.

This way, the server becomes aware of the overall application state and operates in a two-tiered connected way.

The ASP.NET Component Model

The ASP.NET component model provides various building blocks of ASP.NET pages. Basically it is an object model, which describes:

- Server side counterparts of almost all HTML elements or tags, such as <form> and <input>.
- Server controls, which help in developing complex user-interface. For example, the Calendar control or the Gridview control.

ASP.NET is a technology, which works on the .Net framework that contains all web-related functionalities. The .Net framework is made of an object-oriented hierarchy. An ASP.NET web application is made of pages. When a user requests an ASP.NET page, the IIS delegates the processing of the page to the ASP.NET runtime system.

The ASP.NET runtime transforms the .aspx page into an instance of a class, which inherits from the base class page of the .Net framework. Therefore, each ASP.NET page is an object and all its components i.e., the server-side controls are also objects.

Components of .Net Framework 3.5

Before going to the next session on Visual Studio.Net, let us go through at the various components of the .Net framework 3.5. The following table describes the components of the .Net framework 3.5 and the job they perform:

Components and their Description**(1) Common Language Runtime or CLR**

It performs memory management, exception handling, debugging, security checking, thread execution, code execution, code safety, verification, and compilation. The code that is directly managed by the CLR is called the managed code. When the managed code is compiled, the compiler converts the source code into a CPU independent intermediate language (IL) code. A Just In Time(JIT) compiler compiles the IL code into native code, which is CPU specific.

(2) .Net Framework Class Library

It contains a huge library of reusable types. classes, interfaces, structures, and enumerated values, which are collectively called types.

(3) Common Language Specification

It contains the specifications for the .Net supported languages and implementation of language integration.

(4) Common Type System

It provides guidelines for declaring, using, and managing types at runtime, and cross-language communication.

(5) Metadata and Assemblies

Metadata is the binary information describing the program, which is either stored in a portable executable file (PE) or in the memory. Assembly is a logical unit consisting of the assembly manifest, type metadata, IL code, and a set of resources like image files.

(6) Windows Forms

Windows Forms contain the graphical representation of any window displayed in the application.

(7) ASP.NET and ASP.NET AJAX

ASP.NET is the web development model and AJAX is an extension of ASP.NET for developing and implementing AJAX functionality. ASP.NET AJAX contains the components that allow the developer to update data on a website without a complete reload of the page.

(8) ADO.NET

It is the technology used for working with data and databases. It provides access to data sources like SQL server, OLE DB, XML etc. The ADO.NET allows connection to data sources for retrieving, manipulating, and updating data.

(9) Windows Workflow Foundation (WF)

It helps in building workflow-based applications in Windows. It contains activities, workflow runtime, workflow designer, and a rules engine.

(10) Windows Presentation Foundation

It provides a separation between the user interface and the business logic. It helps in developing visually stunning interfaces using documents, media, two and three dimensional graphics, animations, and more.

(11) Windows Communication Foundation (WCF)

It is the technology used for building and executing connected systems.

(12) Windows CardSpace

It provides safety for accessing resources and sharing personal information on the internet.

(13) LINQ

It imparts data querying capabilities to .Net languages using a syntax which is similar to the tradition query language SQL.

ASP.NET provides an abstraction layer on top of HTTP on which the web applications are built. It provides high-level entities such as classes and components within an object-oriented paradigm.

The key development tool for building ASP.NET applications and front ends is Visual Studio. In this tutorial, we work with Visual Studio 2008.

Visual Studio is an integrated development environment for writing, compiling, and debugging the code. It provides a complete set of development tools for building ASP.NET web applications, web services, desktop applications, and mobile applications.

C#.Net

C# is a general-purpose, modern and object-oriented programming language pronounced as “**C sharp**”. It was developed by Microsoft led by Anders Hejlsberg and his team within the .Net initiative and was approved by the European Computer Manufacturers Association (ECMA) and International Standards Organization (ISO). C# is among the languages for Common Language Infrastructure and the current version of C# is version 7.2. C# is a lot similar to Java syntactically and is easy for the users who have knowledge of C, C++ or Java. **A bit about .Net Framework** .Net applications are multi-platform applications and framework can be used from languages like C++, C#, Visual Basic, COBOL etc. It is designed in a manner so that other languages can use it. know more about .Net Framework **Why C#?** C# has many other reasons for being popular and in demand. Few of the reasons are mentioned below:

1. **Easy to start:** C# is a high-level language so it is closer to other popular programming languages like C, C++, and Java and thus becomes easy to learn for anyone.
2. **Widely used for developing Desktop and Web Application:** C# is widely used for developing web applications and Desktop applications. It is one of the most popular languages that is used in professional desktop. If anyone wants to create Microsoft apps, C# is their first choice.
3. **Community:** The larger the community the better it is as new tools and software will be developing to make it better. C# has a large community so the developments are done to make it exist in the system and not become extinct.
4. **Game Development:** C# is widely used in game development and will continue to dominate. C# integrates with Microsoft and thus has a large target audience. The C# features such as Automatic Garbage Collection, interfaces, object-oriented, etc. make C# a popular game developing language.

Features of C#.Net

By design, C# is the programming language that most directly reflects the underlying Common Language Infrastructure (CLI). Most of its intrinsic types correspond to value-types implemented by the CLI framework.

However, the language specification does not state the code generation requirements of the compiler: that is, it does not state that a C# compiler must target a Common Language Runtime, or generate Common Intermediate Language (CIL), or generate any other

specific format. Theoretically, a C# compiler could generate machine code like traditional compilers of C++ or FORTRAN.

Some notable distinguishing features of C# are:

- There are no global variables or functions. All methods and members must be declared within classes. Static members of public classes can substitute for global variables and functions.
- Local variables cannot shadow variables of the enclosing block, unlike C and C++. Variable shadowing is often considered confusing by C++ texts.
- C# supports a strict Boolean data type, `bool`. Statements that take conditions, such as `while` and `if`, require an expression of a type that implements the `true` operator, such as the `boolean` type. While C++ also has a `boolean` type, it can be freely converted to and from integers, and expressions such as `if (a)` require only that `a` is convertible to `bool`, allowing `a` to be an `int`, or a pointer. C# disallows this "integer meaning true or false" approach on the grounds that forcing programmers to use expressions that return exactly `bool` can prevent certain types of common programming mistakes in C or C++ such as `if (a = b)` (use of assignment `=` instead of equality `==`).
- In C#, memory address pointers can only be used within blocks specifically marked as `unsafe`, and programs with `unsafe` code need appropriate permissions to run. Most object access is done through safe object references, which always either point to a "live" object or have the well-defined null value; it is impossible to obtain a reference to a "dead" object (one which has been garbage collected), or to a random block of memory. An `unsafe` pointer can point to an instance of a value-type, array, string, or a block of memory allocated on a stack. Code that is not marked as `unsafe` can still store and manipulate pointers through the `System.IntPtr` type, but it cannot dereference them.
- Managed memory cannot be explicitly freed; instead, it is automatically garbage collected. Garbage collection addresses the problem of memory leaks by freeing the programmer of responsibility for releasing memory which is no longer needed.
- In addition to the `try...catch` construct to handle exceptions, C# has a `try...finally` construct to guarantee execution of the code in the `finally` block.

- Multiple inheritances is not supported, although a class can implement any number of interfaces. This was a design decision by the language's lead architect to avoid complication and simplify architectural requirements throughout CLI.
- C# is more type safe than C++. The only implicit conversions by default are those which are considered safe, such as widening of integers. This is enforced at compile-time, during JIT, and, in some cases, at runtime. There are no implicit conversions between booleans and integers, nor between enumeration members and integers (except for literal 0, which can be implicitly converted to any enumerated type). Any user-defined conversion must be explicitly marked as explicit or implicit, unlike C++ copy constructors and conversion operators, which are both implicit by default.
- Enumeration members are placed in their own scope.
- C# provides properties as syntactic sugar for a common pattern in which a pair of methods, accessor (getter) and mutator (setter) encapsulate operations on a single attribute of a class.
- Full type reflection and discovery is available.
- C# currently (as of version 4.0) has 77 reserved words.
- Checked exceptions are not present in C# (in contrast to Java). This has been a conscious decision based on the issues of scalability and versionability.

Microsoft SQL Server 2008

Organizations today are facing several unique data challenges: the proliferation of data and systems across their enterprise; the need to provide employees, customers, and partners with consistent access to that data; the desire to better equip information workers with meaningful information to drive informed decisions; and the mandate to control costs without sacrificing application availability, security, or reliability.

The next release of SQL Server is designed to help enterprises address these challenges. SQL Server 2008 is Microsoft's next generation data management and analysis solution that will deliver increased security, scalability, and availability to enterprise data and analytical applications while making them easier to create, deploy, and manage. Building on the strengths of SQL Server 2000, SQL Server 2008 will provide an integrated data management and analysis solution that will help organizations of any size to:

1. Build and deploy enterprise applications that are more secure, scalable, and reliable.
2. Maximize the productivity of IT by reducing the complexity of creating, deploying, and managing database applications.
3. Empower developers through a rich, flexible, modern development environment for creating more secure database applications.
4. Share data across multiple platforms, applications, and devices to make it easier to connect internal and external systems.
5. Deliver robust, integrated business intelligence solutions that help drive informed business decisions and increase productivity across your entire organization.
6. Control costs without sacrificing performance, availability, or scalability.

To learn more about the advancements SQL Server 2008 will deliver in three key areas: enterprise data management, developer productivity, and business intelligence. In today's connected world, data and the systems that manage that data must always be available to your users. With SQL Server 2008, users and IT professionals across your organization will benefit from reduced application downtime, increased scalability and performance, and tight security controls.

SQL Server 2008 will include enhancements to enterprise data management in the following areas:

- Availability:

Investments in high availability technologies, additional backup and restore capabilities, and replication enhancements will enable enterprises to build and deploy highly reliable applications.

- Scalability:

Scalability advancements such as partitioning, snapshot isolation, and 64-bit support will enable you to build and deploy your most demanding applications using SQL Server 2008.

- Security:

Enhancements such as "secure by default" settings and an enhanced security model will help provide a high level of security for your enterprise data.

- Manageability:

A new management tool suite, expanded self-tuning capabilities, and a powerful new programming model will increase the productivity of database administrators.

1. XML and Web services:

SQLServer 2008 will support both relational and XML data natively, so enterprises can store, manage, and analyze data in the format that best suits their needs. Support for existing and emerging open standards such as Hypertext Transfer Protocol (HTTP), XML, Simple Object Access Protocol(SOAP), XQuery, and XML Schema Definition (XSD) will also facilitate communication across extended enterprise systems.

Business Intelligence

The challenge and promise of business intelligence revolves around providing employees with the right information, at the right time. Accomplishing this vision demands a business intelligence solution that is comprehensive, secure, integrated with operational systems, and available all day, every day. SQL Server will help companies to achieve this goal with SQL Server 2008. Business intelligence advancements will include:

1. Integrated platform:

SQL Server 2008 will deliver an end-to-end business intelligence platform with integrated analytics including online analytical processing (OLAP); data mining; extract, transformation, and load (ETL) tools; data warehousing; and reporting functionality. This document is developed prior to the product's release to manufacturing, and as such, we cannot guarantee that all details included herein will be exactly as what is found in the shipping product. The information contained in this document represents the current view of Microsoft Corporation on the issues discussed as of the date of publication. Because Microsoft must respond to changing market conditions, it should not be interpreted to be a commitment on the part of Microsoft, and Microsoft cannot guarantee the accuracy of any information presented after the date of publication. The information represents the product at the time this document was printed and should be used for planning purposes only. Information subject to change at any time without prior notice. This document is for informational purposes only.

2. Improved decision making:

Advancements to existing business intelligence features, such as OLAP and data mining, and the introduction of a new reporting server will provide enterprises with the ability to transform information into better business decisions at all organizational levels.

3. Security and availability:

Scalability, availability, and security enhancements will help to provide users with uninterrupted access to business intelligence applications and reports.

4. Enterprise-wide analytical capabilities:

An improved ETL tool will enable organizations to more easily integrate and analyze data from multiple heterogeneous information sources. By analyzing data across a wide array of operational systems, organizations may gain a competitive edge through a holistic understanding of their business.

5. Interoperability:

Through deep support for industry standards, Web services, and the Microsoft .NET Framework, SQL Server 2008 will support interoperability with multiple platforms, applications, and devices.

DeveloperProductivity One of the key barriers to developer productivity has been the lack of integrated tools for database development and debugging. SQL Server 2008 will provide advancements that fundamentally change the way that database applications are developed and deployed. Enhancements for developer productivity will include:

1. Improved tools:

Developers will be able to utilize one development tool for Transact-SQL, XML, Multidimensional Expression (MDX), and XML for Analysis (XML/A). Integration with the Visual Studio® development environment will provide more efficient development and debugging of line-of-business and business intelligence applications.

2. Expanded language support:

With the common language runtime (CLR) hosted in the database engine, developers will be able to choose from a variety of familiar languages to develop database applications, including Transact-SQL, Microsoft Visual Basic .NET, and Microsoft Visual C# .NET.

CHAPTER 3

SYSTEM DESIGN

3.1 INTRODUCTION

Design is the second phase in the system development life cycle. Software design is the first of the three technical activities in the software development process such as design, code writing and testing. During this phase the analyst schedules design activities works with the user to determine the various data inputs to the system, plans how data flow through the system, design required outputs and write program specifications. Again, the analyst's activities on focus on solving a user's problem in logical terms. During this second step analysts employ a variety of tools such as data flow diagram, entity relationship, data dictionaries and grant chart. The system design converts the theoretical solutions introduced by the feasibility study into a logical reality.

3.2.INPUT DESIGN

input design is the process of converting user-oriented input into a computer- based format. The data flow diagram indicates logical data flows, data store source and destinations. Input data are collected and organized into a group of numbered data. Once identified appropriate input data media are selected for processing. The goal of input design is to make data entry as easy and free from errors as much as possible. Proper data validation checking is exercised to correct the mistakes made during data entry.

3.3.OUTPUT DESIGN

Output design is an ongoing activity almost from the beginning of the project. Computer output is the most important and direct source of information to the user. Output design is a process that involves designing necessary outputs that must be given to various user according to their requirements. Efficient output design should improve the system's relationship with the user and help in decision making. The system has got the capability to display standard screen layouts. These layouts should be designed around the output requirements and they must be designed with utmost care and details in the screen layouts must be simple, descriptive and clear to the user. While designing a systems output, the analyst must make several independent decisions. Every system produces reports. No matter what the content of the report is. The following guidelines can be applied to any report.

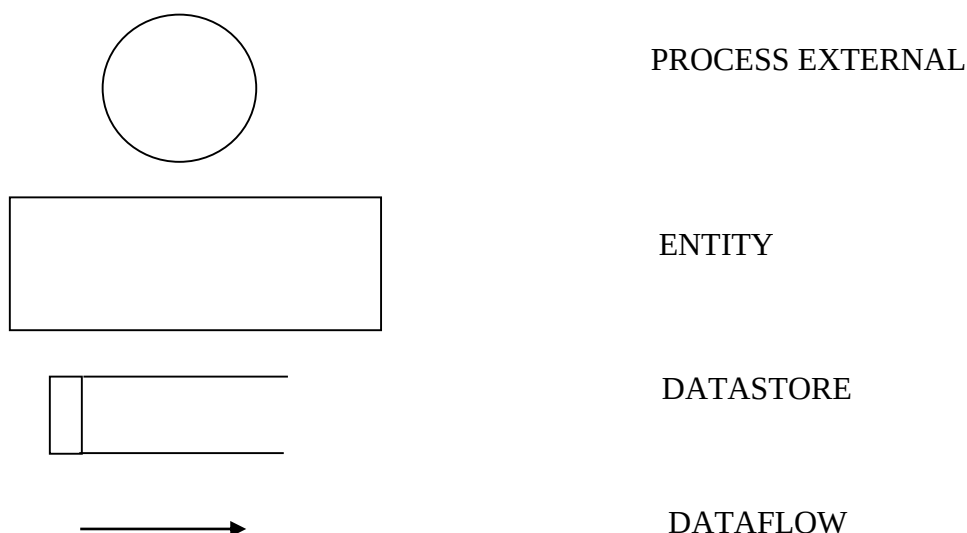
- The information should be clear, accurate, concise and restricted to relevant data
- Reports should have titles and descriptive headings for columns of data, numbered pages and so on.
- The reports content should be in a logical arrangement so that user can easily locate what they need.

3.4 DATA FLOW DIAGRAMS (DFD)

Data flow diagrams were proposed by Larry Constantine. It is a graphical representation of the flow of data through an information system, modeling its process aspects. A DFD is often used as a preliminary step to create an overview of the system, which can later be elaborated. DFDs can also be used for the visualization of data processing.

A DFD shows what kind of information will be input to and output from the system, where the data will come from and go to, and where the data will be stored. It does not show information about the timing of process or information about whether processes will operate in sequence or in parallel.

DATA FLOW NOTATIONS



1.Process/Function

A process represents activities in which data is manipulated by being stored or retrieved or transferred in some way. The process transforms the input data into output data.

2.External Entities

This represents any outside agency, which interact with the system. It represents the source or destination of data for the system under the consideration.

3.Data Stores/Database

A data store represents a logical life. Data store is represented by open ended rectangle. Each data store is connected to a process by means of a data flow symbol.

4. Data Flows

The data flow represents in an interface among different components in a DFD. It represents flow of data between two processes or between a process and an external entity or between a process and data store.

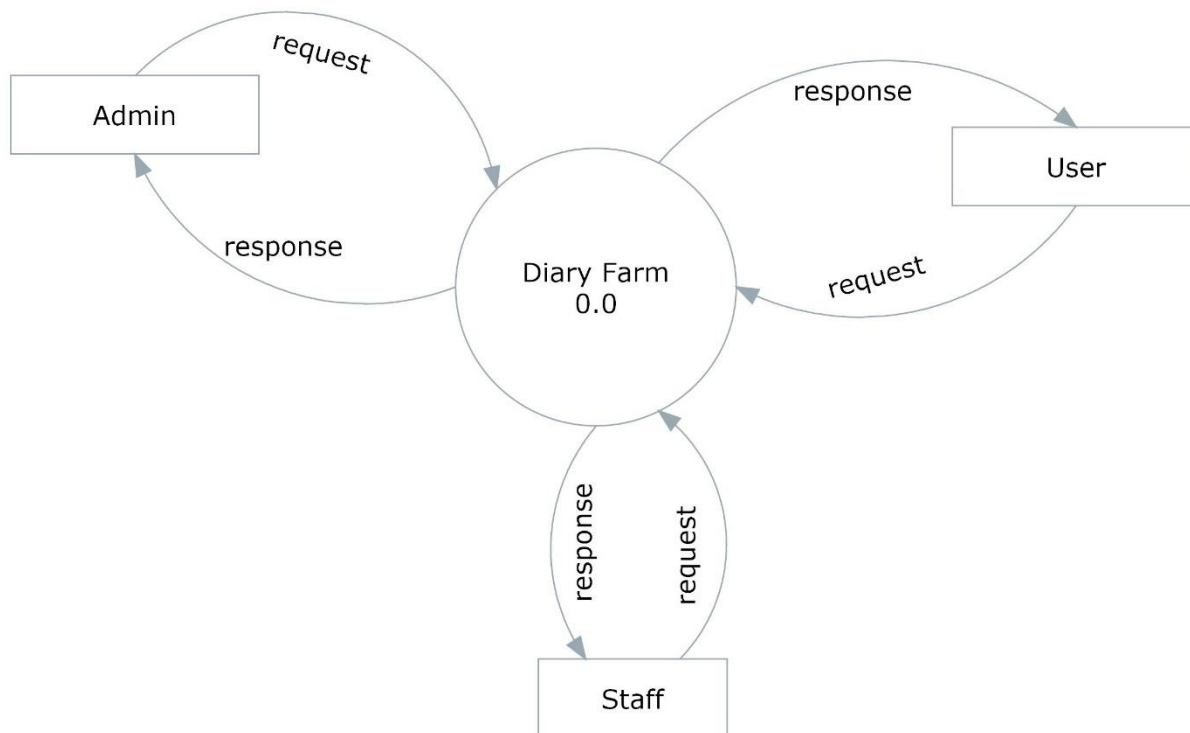
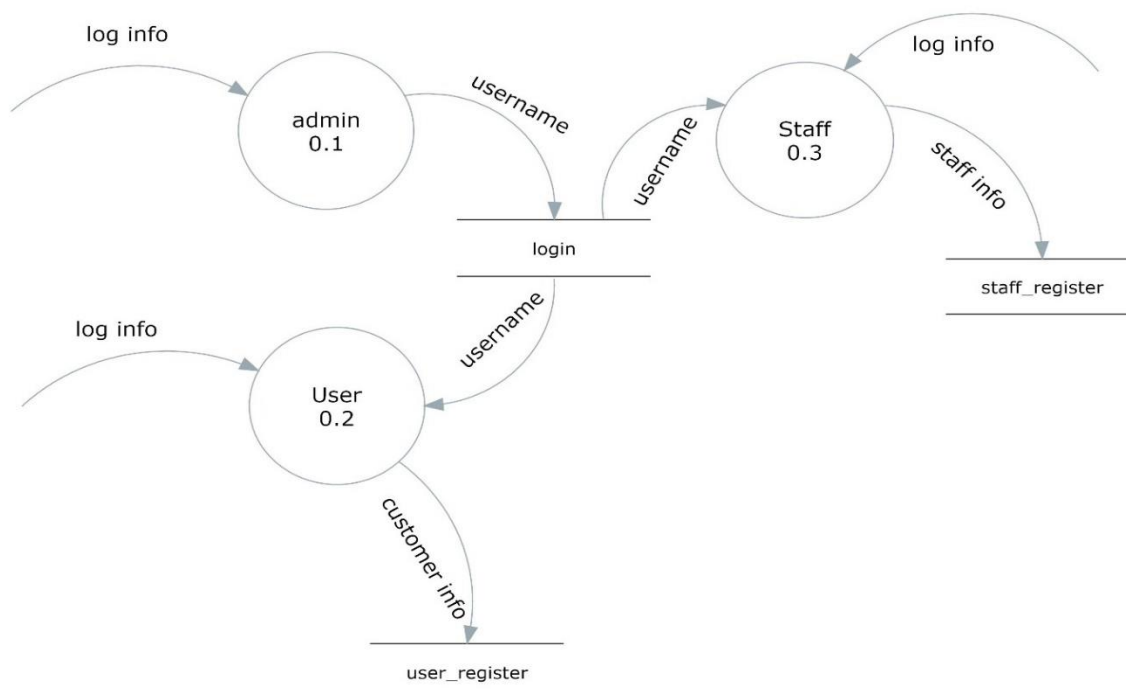
A few guidelines to aid the derivation of a DFD :

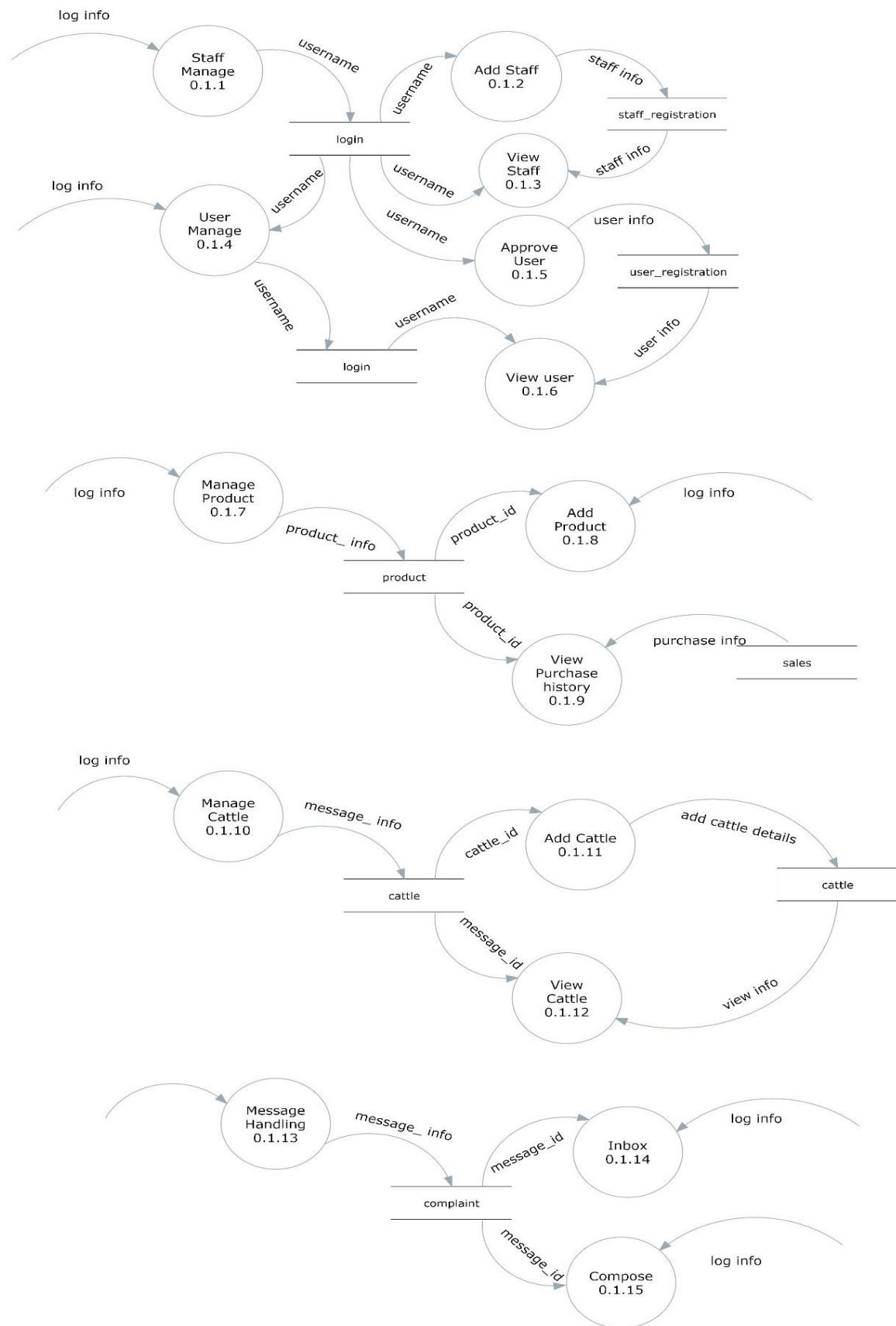
- The level 0 DFD should depict the software/system as a single bubble.
- Primary input and output should be noted.
- Refinement should begin by isolating candidate processes, data objects and the stores to be represented in the next level.
- All arrows and bubbles should be labeled with a meaningful name.
- Information flow continuity must be maintained from level to level.
- One bubble at a time should be refined.

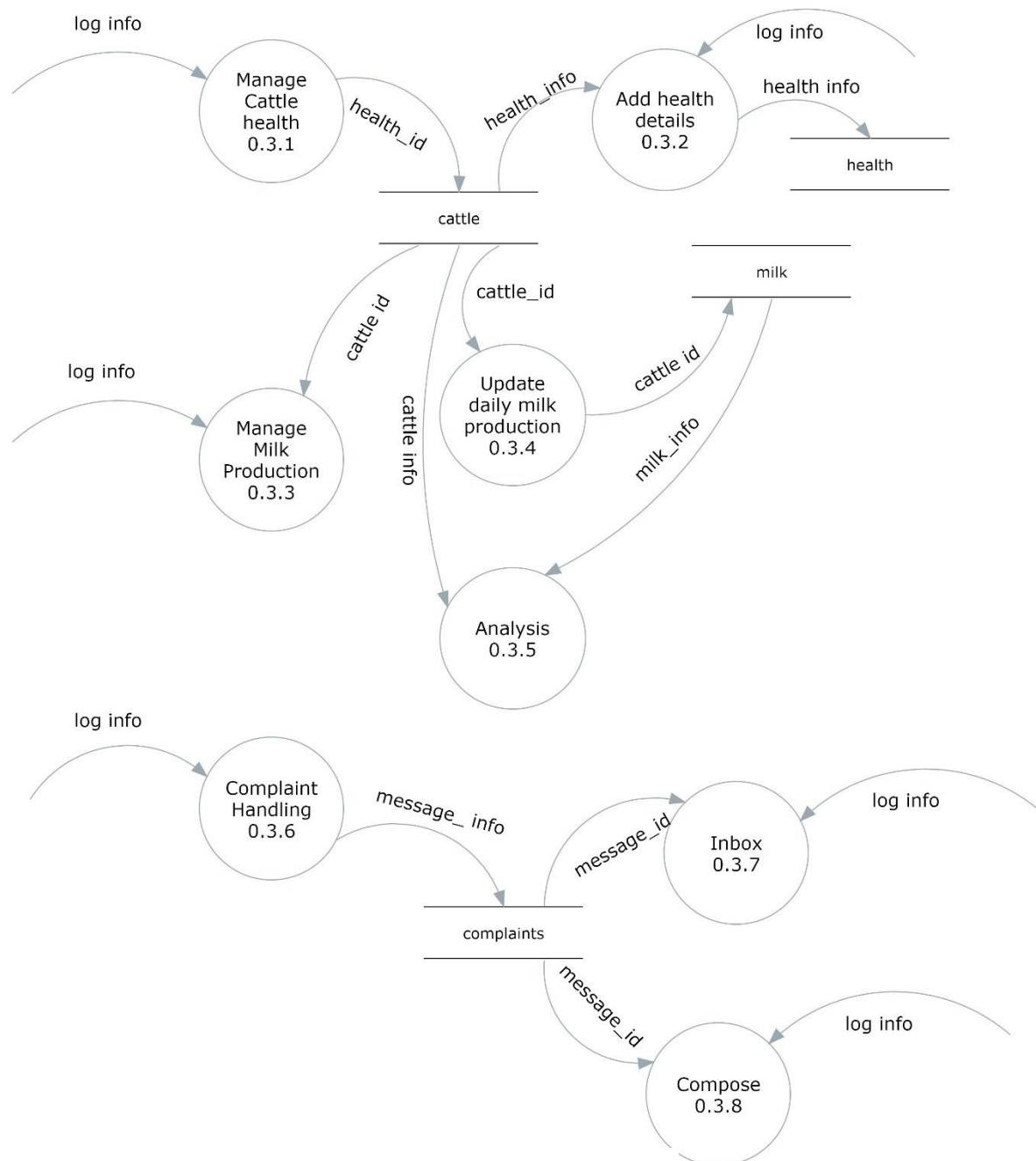
The level 0 DFD is expanded to level 1 DFD. The process represented in the level 1 DFD can be further refined to level 2 DFD. Information flow continuity is maintained between levels. The refinement continues until each bubble performs a simple function. That is, until the process represented by the bubble performs a function that would be easily implemented as a program.

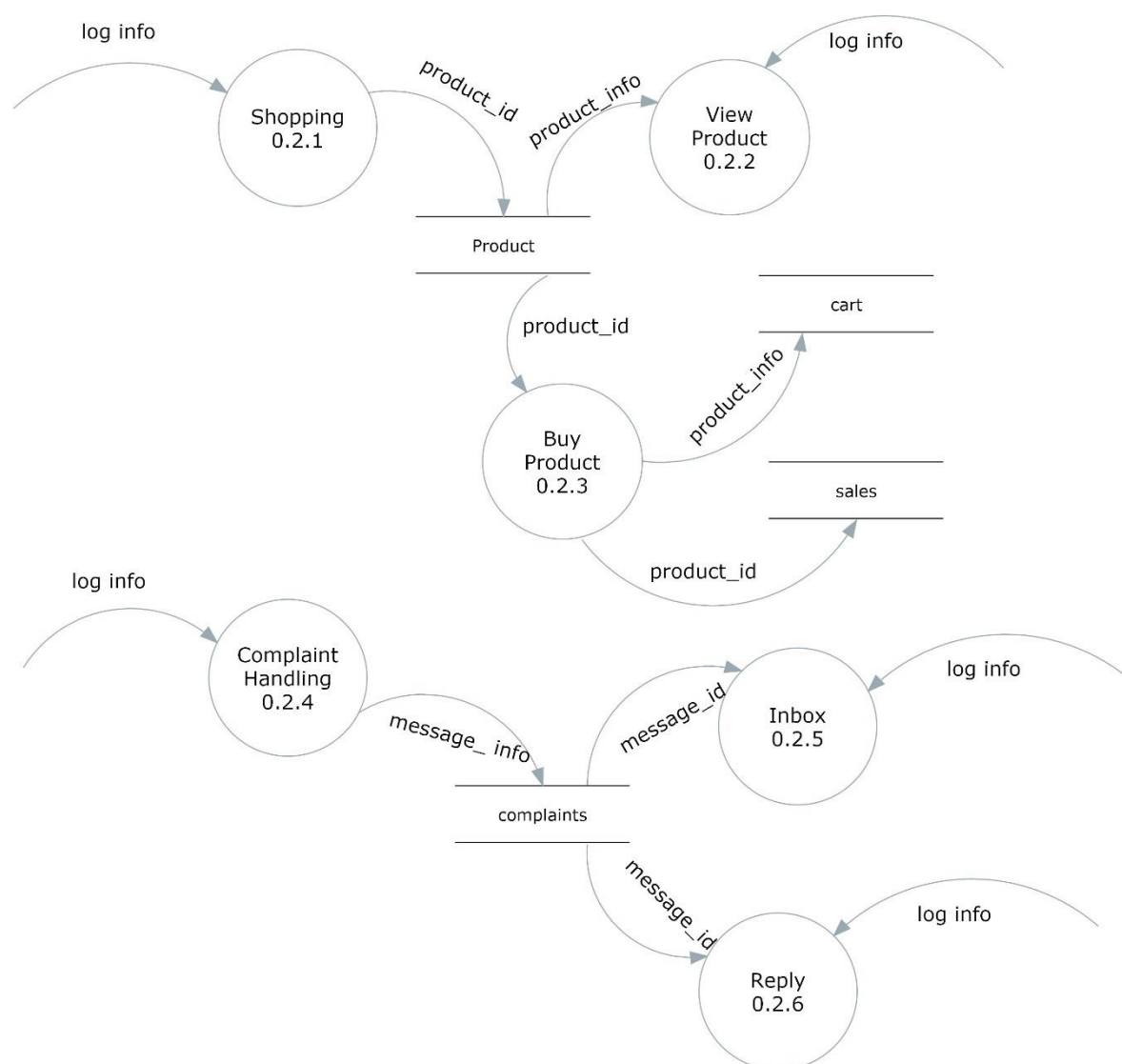
The DFD serves two purposes :

- To provide an indication of how data is transformed as they move through the system, and to depict the functions (and sub functions) that transforms the data flow.
- DFD provide additional information that is used during the analysis of the information domain and serves as a basis for the modeling of functions.

LEVEL ZERO / Context Diagram**Fig 3.4.1****Level 1****Fig 3.4.2**

Level 2 Admin**Fig 3.4.3**

Level 2 Staff**Fig 3.4.4**

Level 2 User**Fig 3.4.5****3.5 STRUCTURE CHART**

A structure chart is a [chart](#) which shows the breakdown of a system to its lowest manageable levels. They are used in [structured programming](#) to arrange program modules into a tree. Each module is represented by a box, which contains the module's name. The tree structure visualizes the relationships between modules.

A structure chart is a [top-down modular design](#) tool, constructed of squares representing the different modules in the [system](#), and lines that connect them. The lines represent the connection between activities and sub activities as they are used in [organization charts](#).

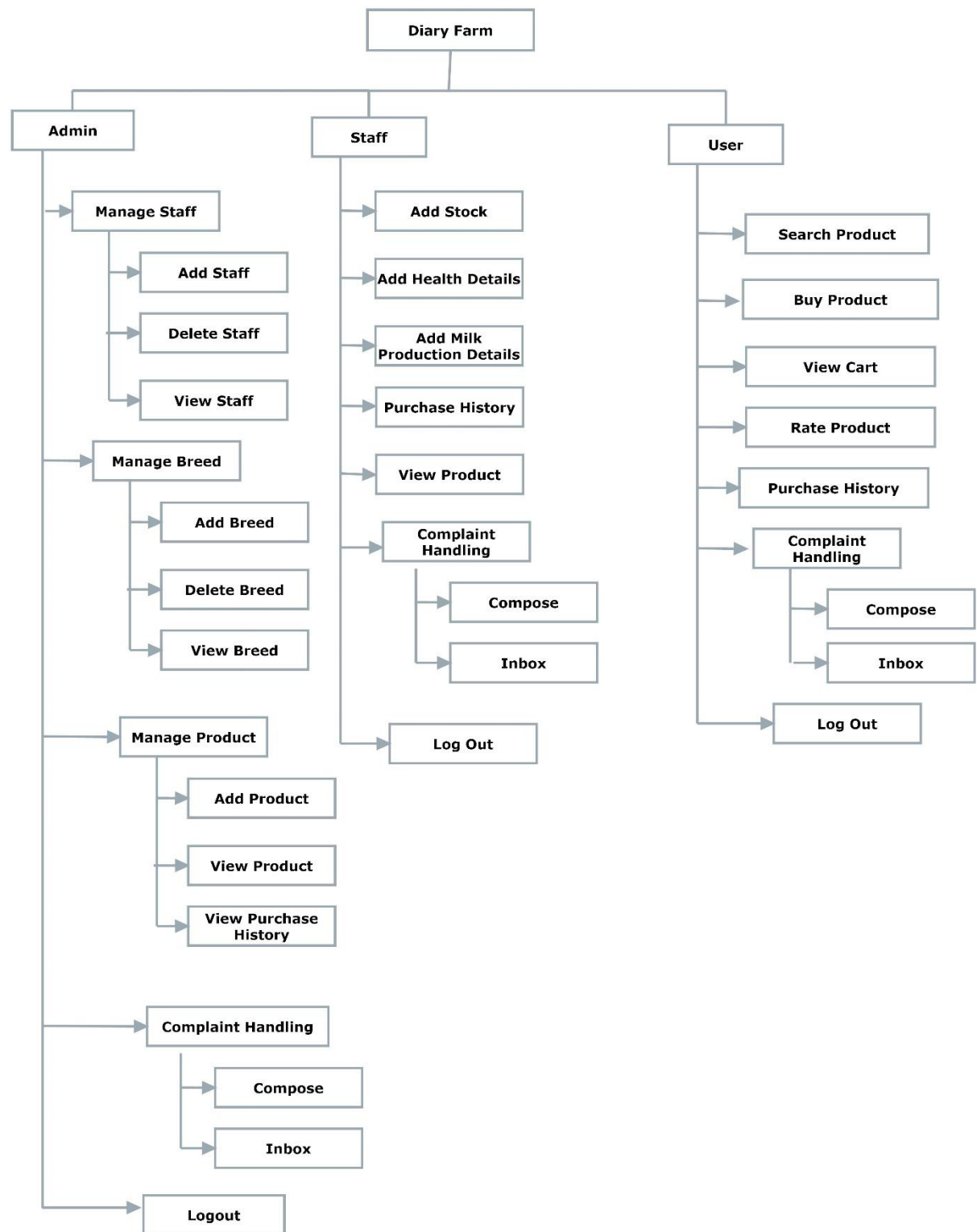


Fig 3.5

3.6 DATA BASE DESIGN

A relational database is a collection of data items organized as a set of formally-described tables from which data can be accessed or reassembled in many different ways without having to reorganize the database tables. The relational database was invented by E. F. Codd at IBM in 1970.

A database is an organized collection of data. The data is typically organized to model aspects of reality in a way that supports processes requiring information. It refers to a set of related data and the way it is structured or organized.

Database design is the process of producing a detailed data model of a database. The process of doing database design generally consists of a number of steps which will be carried out by the database designer. Usually, the designer must:

- Determine the relationships between the different data elements.
- Superimpose a logical structure upon the data on the basis of these relationships.

Normalization

Normalization is a design technique that is widely used as a guide in designing relational databases. It is the process of decomposing the attributes in an application, which results in a set of tables with very simple structure.

There are two goals of the normalization process:

- Eliminate redundant data
- Ensure data dependencies

The database community has developed a series of guidelines for ensuring that databases are normalized. These are referred to as normal forms. It is referred as 1NF, 2NF, and 3NF along with BCNF.

Constraints in creating a table:

- **Candidate Key** : A candidate key is a combination of one or more columns, the value of which uniquely identifies each row of a table.
- **primary Key** : The primary key of a table is one of the candidate keys that you give some characteristics. We can have only one primary key, and a primary key column cannot contain NULLs.
- **Foreign Key** : A foreign key is a combination of columns with values based on primary key values from another table. A foreign key constraint, also known as Referential Integrity constraint, specifies that the values of the foreign key

correspond to actual values of the primary key in the other table.

- **CHECK Constraint :** Many columns must have values that are within a certain range or that satisfy certain conditions. With a CHECK constraint, we can specify an expression that must always be true for every row in the table.

Rules of Data Normalization

- **1 NF :**
1 NF specifies eliminate repeating groups. It make a separate table for each set of related attributes, and give each table a primary key.
- **2 NF :**
2 NF specifies eliminate redundant data. If an attribute depends on only one part of a multi-valued key, remove it to a separate table.
- **3 NF :**
3NF specifies eliminate columns not dependent on key. If attributes do not contribute to a description of the key, remove them to a separate table.
- **BCNF :**
It means Boyce-Codd Normal Form. If there are non-trivial dependencies between candidate key attributes, separate them out into distinct tables.

Table 3.6.1 tbl_Login

Field Name	Data Type	Size	Constraint	Description
username	varchar	20	Primary Key	Username
password	varchar	20	NOT NULL	Password
uid	int	4	NOT NULL	User Identification
status	int	3	NOT NULL	Status

Table 3.6.2 tbl_user_register

Field Name	Data Type	Size	Constraint	Description
uid	int	4	Primary Key	User Identification
uuname	varchar	20	Foreign Key	User username
name	varchar	20	NOT NULL	User Name
email	varchar	30	NOT NULL	User Email ID
pwd	varchar	20	NOT NULL	User Password
address	varchar	100	NOT NULL	User address

location	varchar	50	NOT NULL	User location
contact	varchar	10	NOT NULL	User contact
dob	varchar	20	NOT NULL	User date of Birth
status	int	2	NOT NULL	Status

Table 3.6.3 tbl_staff_register

Field Name	Data Type	Size	Constraint	Description
stid	int	4	Primary Key	Staff Identification
suname	varchar	20	Foreign Key	Staff username
name	varchar	20	NOT NULL	Staff Name
email	varchar	30	NOT NULL	Staff Email ID
pwd	varchar	20	NOT NULL	Staff Password
address	varchar	100	NOT NULL	Staff address
location	varchar	50	NOT NULL	location
contact	varchar	10	NOT NULL	contact
status	int	2	NOT NULL	Status

Table 3.6.4 tbl_cart

Field Name	Data Type	Size	Constraint	Description
cartid	int	4	Primary Key	Cart Identification
pid	varchar	20	Foreign Key	Product ID
cuname	varchar	20	Foreign Key	Customer Username
Pmimage	varchar	100	NOT NULL	Image
pname	varchar	30	NOT NULL	Product Name
Price	int	10	NOT NULL	Price
Quantity	int	10	NOT NULL	Quantity
status	int	2	NOT NULL	Status

Table 3.6.5 tbl_complaints

Field Name	Data Type	Size	Constraint	Description
compid	int	4	Primary Key	Mail ID
mdate	varchar	20	NOT NULL	Mail Date
mfrom	varchar	20	Foreign Key	Mail From
mtto	varchar	50	Foreign Key	Mail To
subject	varchar	20	NOT NULL	Subject
attachment	varchar	10	NOT NULL	Attachment
message	varchar	50	NOT NULL	Message
status	varchar	12	NOT NULL	Status

Table 3.6.6 tbl_Sales

Field Name	Data Type	Size	Constraint	Description
Salesid	int	4	Primary Key	Sales ID
Uname	varchar	20	NOT NULL	Username
Quantity	Varchar	4	NOT NULL	Quantity
Amount	Varchar	20	NOT NULL	Amount
Pdate	Varchar	20	NOT NULL	Purchase Date
Ddate	Varchar	20	NOT NULL	Delivery Date
status	Int	4	NOT NULL	Status

Table 3.6.7 tbl_Breed

Fieldname	Datatype	Size	Constraints	Description
bid	integer	20	Primary key	Breed id
gender	varchar	10	Not null	gender
name	varchar	50	Not null	Cattle name
milk	varchar	50	Not null	Milk
lifespan	integer	20	Not null	Cattle lifepsan
description	varchar	300	Not null	Description
orgin	varchar	100	Not null	Cattle orgin
weight	integer	20	Not null	Weight

Table 3.6.8 tbl_Health

Fieldname	Datatype	Size	Constraints	Description
hid	integer	11	Primary key	Health id
cid	integer	11	Foreign key	Cattle id
status	varchar	20	Not null	Health status
Weight	integer	20	Not null	Weight
doctor	varchar	50	Not null	Doctor name
description	varchar	300	Not null	Description
date	integer	25	Not null	Date

Table 3.6.9 tbl_Milk

Field Name	Data Type	Size	Constraint	Description
mid	int	4	Primary Key	Milk ID
bid	varchar	20	Foreign key	Breed ID
amount	Varchar	4	NOT NULL	Quantity of milk
date	Varchar	20	NOT NULL	date
time	Varchar	20	NOT NULL	Time slot
status	Int	4	NOT NULL	Status

Table 3.6.10 tbl_Product

Field Name	Data Type	Size	Constraint	Description
Pcode	int	4	Primary Key	Product code
Ptitle	Varchar	20	NOT NULL	Product title
pprice	Varchar	50	NOT NULL	Product price
pdate	Varchar	20	NOT NULL	Product date
Pimage	Varchar	20	NOT NULL	Product image
Pimage1	Varchar	20	NOT NULL	Product sub image1
Pimage2	Varchar	20	NOT NULL	Product sub image2
Pimage3	Varchar	20	NOT NULL	Product sub image3
Rating	int	4	NOT NULL	Rating
status	int	4	NOT NULL	Status

3.7 ER-Diagram

Database designs also include ER (Entity-relationship model) diagrams. An ER diagram is a diagram that helps to design databases in an efficient way. Attributes in ER diagrams are usually modeled as an oval with the name of the attribute, linked to the entity or relationship that contains the attribute. Within the relational model the final step can generally be broken down into two further steps, that of determining the grouping of information within the system, generally determining what are the basic objects about which information is being stored, and then determining the relationships between these groups of information, or objects. This step is not necessary with an Object database

The E-R Model: The enterprise is viewed as set of

- Entities
- Relationships among entities
- Symbols used in E-R Diagram
- Entity – rectangle
- Attribute – oval
- Relationship – diamond
- Link – line

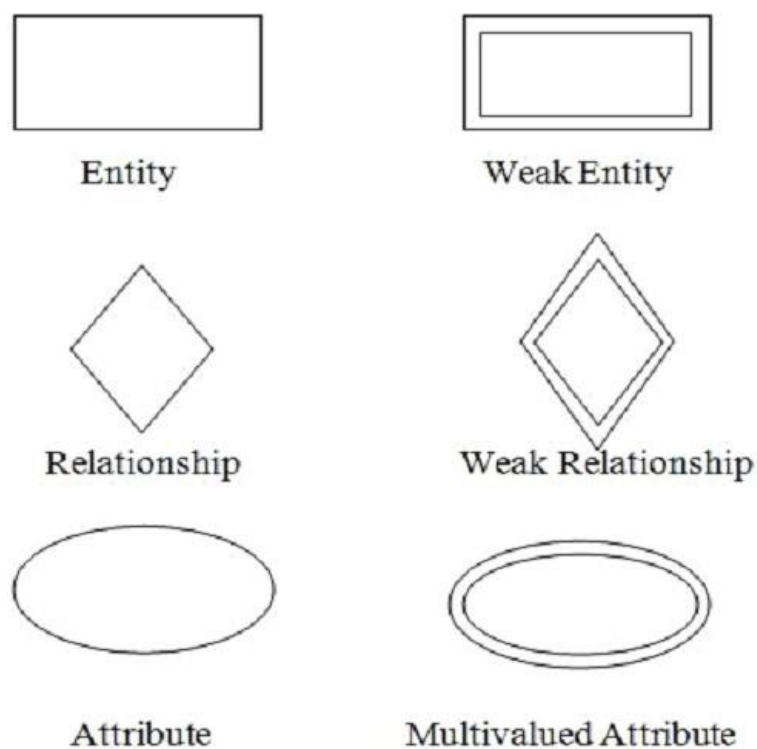


Fig: Symbols of ER diagram

Entity

An entity can be a person, place, event, or object that is relevant to a given system. For example, a school system may include students, teachers, major courses, subjects, fees, and other items. Entities are represented in ER diagrams by a rectangle and named using singular nouns.

Weak Entity

A weak entity is an entity that depends on the existence of another entity. In more technical terms it can be defined as an entity that cannot be identified by its own attributes. It uses a foreign key combined with its attributes to form the primary key. An entity like order item is a good example for this. The order item will be meaningless without an order so it depends on the existence of the order.

Attribute

An attribute is a property, trait, or characteristic of an entity, relationship, or another attribute. For example, the attribute Inventory Item Name is an attribute of the entity Inventory Item. An entity can have as many attributes as necessary. Meanwhile, attributes can also have their own specific attributes. For example, the attribute “customer address” can have the attributes number, street, city, and state. These are called composite attributes. Note that some top level ER diagrams do not show attributes for the sake of simplicity. In those that do, however, attributes are represented by oval shapes.

Relationship

A relationship describes how entities interact. For example, the entity “carpenter” may be related to the entity “table” by the relationship “builds” or “makes”. Relationships are represented by diamond shapes and are labeled using verbs.

Multivalued Attribute

If an attribute can have more than one value it is called a multivalued attribute. It is important to note that this is different to an attribute having its own attributes. For example a teacher entity can have multiple subject values.

Derived Attribute

An attribute based on another attribute. This is found rarely in ER diagrams. For example, for a circle the area can be derived from the radius

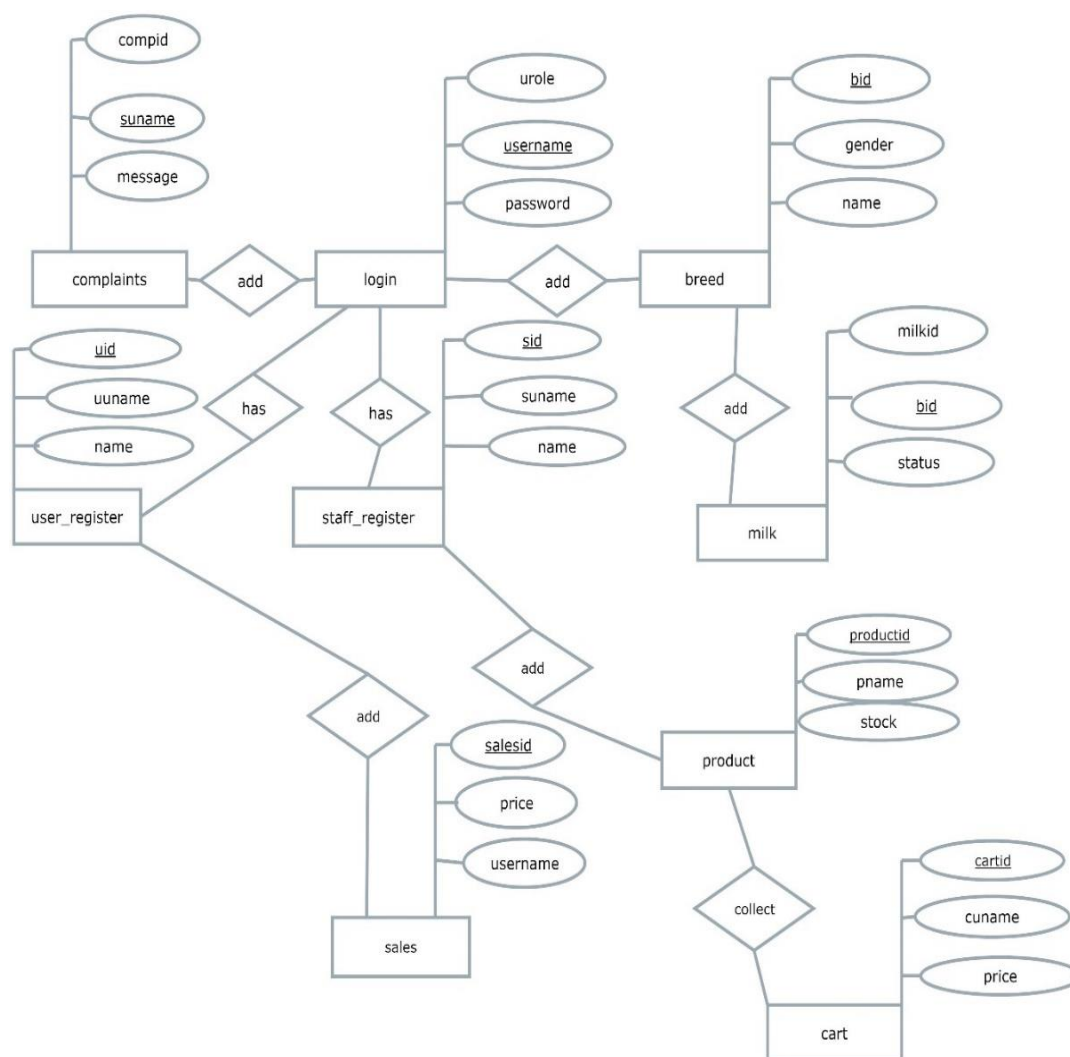


Fig 3.7

3.8 USE CASE DIAGRAM

To model a system the most important aspect is to capture the dynamic behavior. To clarify a bit in details, dynamic behavior means the behavior of the system when it is running or operating. So only static behavior is not sufficient to model a system rather dynamic behavior is more important than static behavior. In UML there are five diagrams available to model dynamic nature and use case diagram is one of them. These internal and external agents are known as actors. So use case diagrams are consists of actors, use cases and their relationships. The diagram is used to model the system/subsystem of an application. A

single use case diagram captures a particular functionality of a system. The purposes of use case diagrams can be as follows:

- Used to gather requirements of a system.
- Used to get an outside view of a system.
- Identify external and internal factors influencing the system.

The main purpose of a use case diagram is to show what system functions are performed for which actors. Roles of the actors in the system can be depicted. Use cases are used in almost every project. They are helpful in exposing requirements and planning the project. The two main components of a use case diagram are use cases and actors.

USE CASE: ADMIN

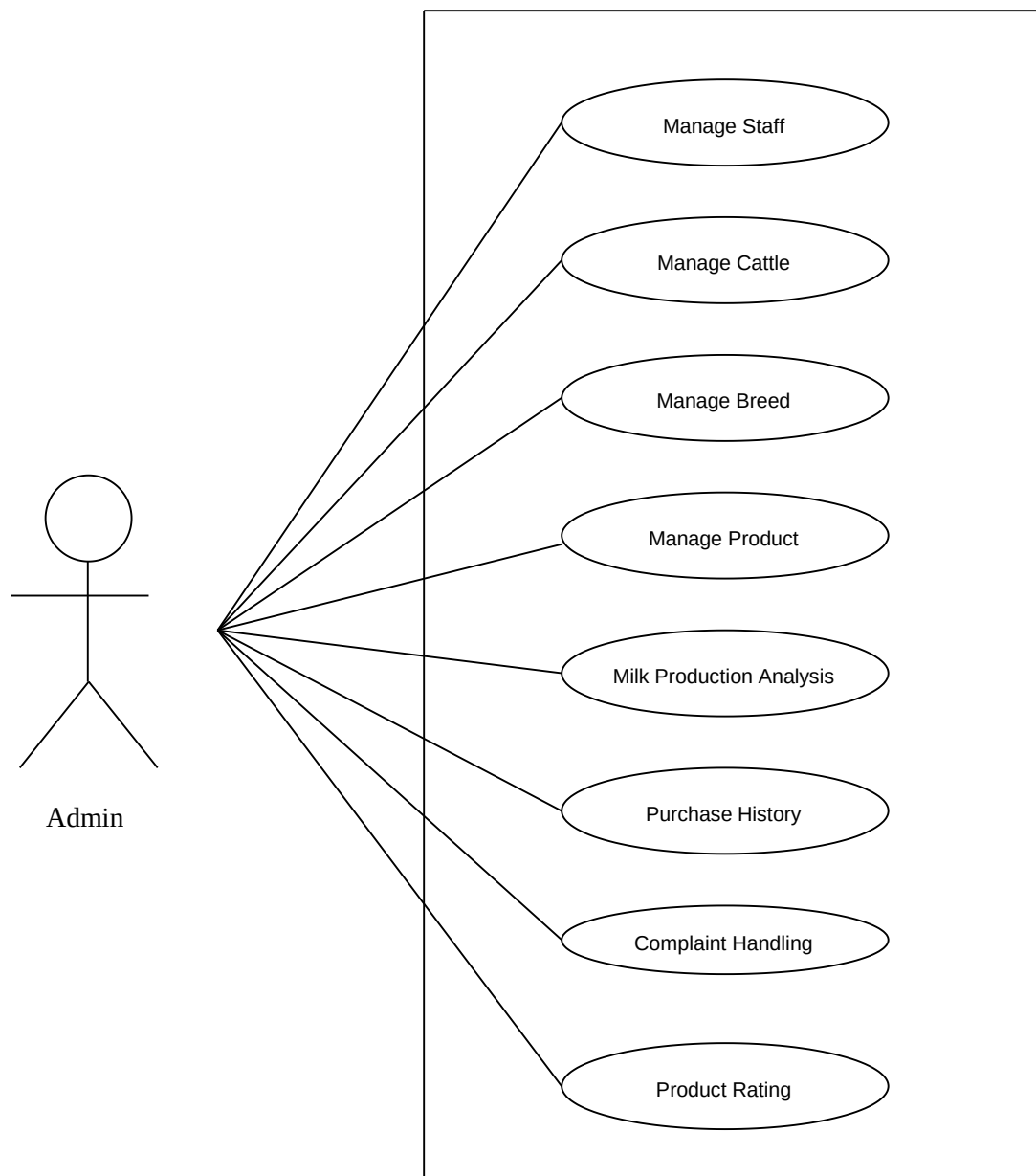
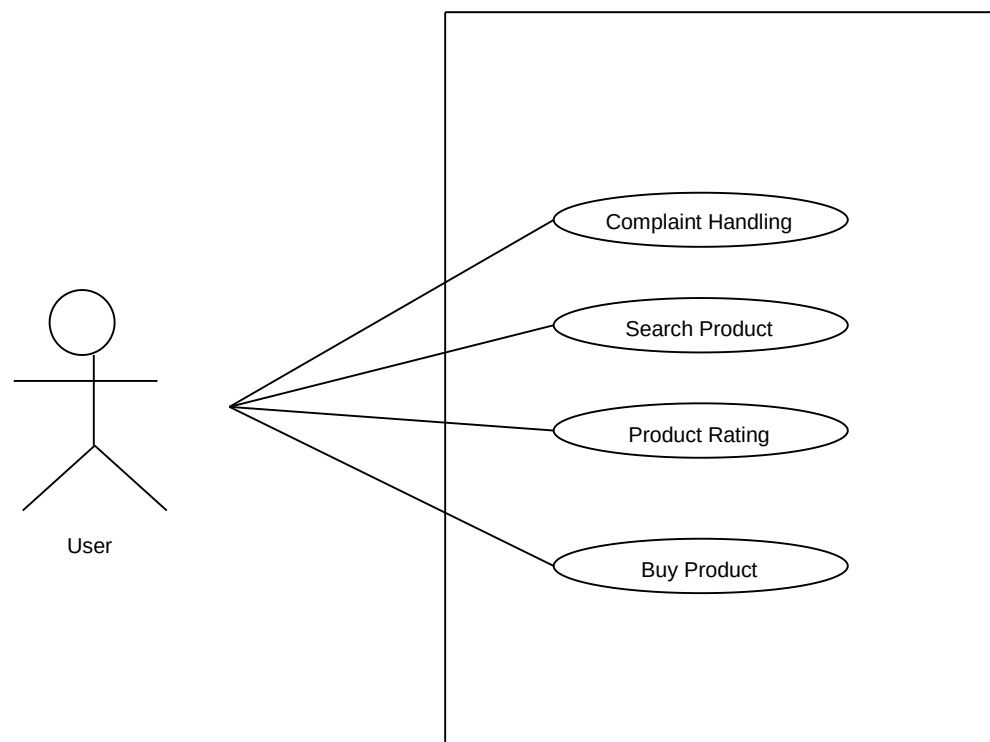


Fig 3.8.1

USE CASE : STAFF**Fig 3.8.2****USE CASE : USER****Fig 3.8.3**

CHAPTER 4

SYSTEM CODING

Login

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

public partial class Login : System.Web.UI.Page
{
    DatabaseConnection ob = new DatabaseConnection();
    protected void Button1_Click(object sender, EventArgs e)
    {
        Session["uname"] = TextBox1.Text;
        Session["pwd"] = TextBox8.Text;
        if (TextBox8.Text == "" || TextBox1.Text == "")
        {
            Label3.Visible = true;
            Label3.ForeColor = System.Drawing.Color.Red;
            Label3.Text = "Please Enter all fields correctly!";
        }
        else
        {
            ob.dr = ob.ret_dr("select * from Login where uname='" + TextBox1.Text + "' and
pass='" + TextBox8.Text + "'");
            if (ob.dr.Read())
            {
                if (ob.dr[3].ToString() == '0'.ToString())
                {
                    Session["uid"] = TextBox1.Text;
                    Session["uname"] = TextBox1.Text;
                    Response.Redirect("UserSearchProduct.aspx", true);
                }
                else if (ob.dr[3].ToString() == '2'.ToString())
                {
                    Session["suname"] = TextBox1.Text;
                    //Session["shopid"] = ob.dr[0].ToString();
                    Response.Redirect("StaffAddStock.aspx", true);
                }
                else if (ob.dr[3].ToString() == '1'.ToString())
                {
                    Session["uid"] = TextBox1.Text;
                    Response.Redirect("AdminAddBreed.aspx", true);
                }
            }
        }
    }
}
```

```

        else
        {
            Response.Write("<script type = 'text/javascript'>alert('Invalid
User');</script>");
            TextBox1.Text = "";
            TextBox8.Text = "";
        }
    }
}
}
}

```

Add Cattle

```

using System;
using System.Collections.Generic;
using System.Globalization;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

public partial class AdminAddBreed : System.Web.UI.Page
{
    DatabaseConnection ob = new DatabaseConnection();
    DatabaseConnection ob1 = new DatabaseConnection();
    public static string aa = "";
    public static string formattedDate;
    protected void Page_Load(object sender, EventArgs e)
    {
        if (!IsPostBack)
        {
            DropDownList7.Items.Insert(0, new ListItem("SELECT", "0"));
            DropDownList7.Items.Insert(1, new ListItem("Healthy", "1"));
            DropDownList7.Items.Insert(2, new ListItem("Sick", "2"));
            DropDownList7.Items.Insert(3, new ListItem("Recovering", "3"));
            DropDownList7.Items.Insert(4, new ListItem("Injured", "4"));

        }

        TextBox1.Text = getid();
    }
    public string getid()
    {
        int c = 0;
        ob.dr = ob.ret_dr("select isnull(max(cowid),300)+1 from cattle");
        if (ob.dr.Read())
        {
            c = Convert.ToInt32(ob.dr[0].ToString());
        }
        return c.ToString();
    }
}

```

```

    }
    protected void DropDownList7_SelectedIndexChanged(object sender, EventArgs e)
    {

    }

    protected void Button1_Click(object sender, EventArgs e)
    {
        if (TextBox1.Text == "" || TextBox2.Text == "" ||
        DropDownList7.SelectedItem.ToString() == "Select")
        {
            Label11.Visible = true;
            Label11.ForeColor = System.Drawing.Color.Red;
            Label11.Text = "Please Enter all fields correctly!";
        }
        else
        {
            string aa = "NA";

            //string formattedDate = TextBox8.Text.ToString("dd-MM-yyyy", null);
            ob.exec("insert into cattle values('" + TextBox1.Text + "','" + TextBox2.Text +
            "','" + TextBox3.Text + "','" + TextBox8.Text + "','" +
            DropDownList7.SelectedItem.ToString() + "','0,'" + TextBox7.Text + "','" + TextBox8.Text
            + "')");
            Label11.Visible = true;
            ScriptManager.RegisterStartupScript(this, this.GetType(), "message",
            "alert('Details Added Successfully');", true);
            Label11.ForeColor = System.Drawing.Color.Green;
            Label11.Text = "Your request has been approved!";
            TextBox1.Text = "";
            TextBox2.Text = "";
            TextBox7.Text = "";
            TextBox3.Text = "";

            TextBox1.Text = getid();

            DropDownList7.SelectedIndex = 0;

        }
    }
}

```

Admin Add Product

```

using System;
using System.Collections.Generic;
using System.Drawing.Imaging;
using System.Drawing;
using System.IO;
using System.Linq;
using System.Web;

```

```
using System.Web.UI;
using System.Web.UI.WebControls;
using System.Xml.Linq;

public partial class AdminAddProduct : System.Web.UI.Page
{
    DatabaseConnection ob = new DatabaseConnection();
    public static string document2 = "", document3 = "", document4 = "", document21 =
    "", document31 = "", document41 = "", document22 = "", document32 = "", document42
    = "", document23 = "", document33 = "", document43 = "", img2 = "";
    DatabaseConnection ob1 = new DatabaseConnection();
    public static int sid;
    public static string duname;
    protected void Page_Load(object sender, EventArgs e)
    {
        getid();
        stockgetid();
        duname = "admin";
        TextBox15.Text = DateTime.Now.ToShortDateString();
    }
    void getid()
    {
        ob.dr = ob.ret_dr("select isnull(max(pcode),500)+1 from product");
        if (ob.dr.Read())
        {
            TextBox8.Text = ob.dr[0].ToString();
        }
    }
    void stockgetid()
    {
        ob.dr = ob.ret_dr("select isnull(max(stockid),900)+1 from stock");
        if (ob.dr.Read())
        {
            sid = Convert.ToInt32(ob.dr[0].ToString());
            // TextBox2.Text = sid.ToString();
        }
    }
    protected void Button1_Click(object sender, EventArgs e)
    {
        if (TextBox10.Text == "" || TextBox9.Text == "" || TextBox13.Text == "" ||
        TextBox14.Text == "" || TextBox15.Text == "" || TextBox6.Text == "")
        {
            Label11.Visible = true;
            Label11.ForeColor = System.Drawing.Color.Red;
            Label11.Text = "Please Enter all fields correctly!";
        }
        else
        {
            string add = "admin";
```

```

document4 = "mdesign/" + document3;
document41 = "sdesign1/" + document31;
document42 = "sdesign2/" + document32;
document43 = "sdesign3/" + document33;
string str1 = "insert into product values(" + TextBox8.Text + "," +
TextBox3.Text + "," + TextBox6.Text + "," + TextBox10.Text + "," + TextBox9.Text
+ "),(" + TextBox13.Text + "," + TextBox14.Text + "," + TextBox15.Text + "," +
document4 + "," + document41 + "," + document42 + "," + document43 + ",0,0)";
ob.exec(str1);
ob1.exec("insert into stock values(" + sid + "," + TextBox8.Text + "," +
TextBox3.Text + "," + add + "," + TextBox10.Text + "," + TextBox14.Text + "," +
TextBox15.Text + ",0,0,0)");

Label11.Visible = true;
Label11.ForeColor = System.Drawing.Color.Green;
Label11.Text = "Data stored successfully !";

TextBox3.Text = "";

//TextBox6.Text = "";
ScriptManager.RegisterStartupScript(this, this.GetType(), "message",
>alert('Product Added Successfully');", true);
//Response.Redirect("ShopAdminAddStock.aspx");

}
}

protected void FileUpload1_Load(object sender, EventArgs e)
{
    try
    {
        document2 = Server.MapPath("~/mdesign/" + "/");
        img2 = Server.MapPath("~/ImgProduct/" + "/");
        if (!Directory.Exists(document2))
            Directory.CreateDirectory(document2);
        if (FileUpload1.HasFile)
        {
            try
            {
                // document2 = document2 + FileUpload1.FileName.Trim(' ');
                string extension = Path.GetExtension(FileUpload1.PostedFile.FileName);

                FileUpload1.SaveAs(document2 + TextBox8.Text + extension);
                document3 = TextBox8.Text + extension;
            }
            catch (Exception ex)
            { }
        }
    }
}

```

```
    }
  }
  catch (Exception ex)
  {
  }
}

protected void FileUpload2_Load(object sender, EventArgs e)
{
  try
  {
    document21 = Server.MapPath("~/sdesign1/" + "/");
    if (!Directory.Exists(document21))
      Directory.CreateDirectory(document21);
    if (FileUpload2.HasFile)
    {
      try
      {
        // document2 = document2 + FileUpload1.FileName.Trim(' ');
        string extension = Path.GetExtension(FileUpload2.PostedFile.FileName);
        FileUpload2.SaveAs(document21 + TextBox8.Text + extension);
        document31 = TextBox8.Text + extension;
      }
      catch (Exception ex)
      { }
    }
  }
  catch (Exception ex)
  {
  }
}

protected void FileUpload3_Load(object sender, EventArgs e)
{
  try
  {
    document22 = Server.MapPath("~/sdesign2/" + "/");
    if (!Directory.Exists(document22))
      Directory.CreateDirectory(document22);
    if (FileUpload3.HasFile)
    {
      try
      {
        // document2 = document2 + FileUpload1.FileName.Trim(' ');
        string extension = Path.GetExtension(FileUpload3.PostedFile.FileName);
        FileUpload3.SaveAs(document22 + TextBox8.Text + extension);
        document32 = TextBox8.Text + extension;
      }
    }
  }
}
```

```
        catch (Exception ex)
        { }

    }
}
catch (Exception ex)
{
}
}

protected void FileUpload4_Load(object sender, EventArgs e)
{
    try
    {
        document23 = Server.MapPath("~/sdesign3/" + "/");
        if (!Directory.Exists(document23))
            Directory.CreateDirectory(document23);
        if (FileUpload3.HasFile)
        {
            try
            {
                // document2 = document2 + FileUpload1.FileName.Trim(' ');
                string extension = Path.GetExtension(FileUpload4.PostedFile.FileName);
                FileUpload4.SaveAs(document23 + TextBox8.Text + extension);
                document33 = TextBox8.Text + extension;
            }
            catch (Exception ex)
            { }
        }
    }
    catch (Exception ex)
    {
    }
}
}
```

Admin Add Staff

```
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Linq;
using System.Security.Cryptography;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

public partial class AdminAddStaff : System.Web.UI.Page
{
    DatabaseConnection ob = new DatabaseConnection();
```

```

DatabaseConnection ob1 = new DatabaseConnection();
protected void Page_Load(object sender, EventArgs e)
{
    if (!IsPostBack)
    {
        DropDownList7.Items.Insert(0, new ListItem("SELECT", "0"));
        DropDownList7.Items.Insert(1, new ListItem("Male", "1"));
        DropDownList7.Items.Insert(2, new ListItem("Female", "2"));
        DropDownList7.Items.Insert(3, new ListItem("Transgender", "3"));

    }

    TextBox1.Text = getid();
}
public string getid()
{
    int c = 0;
    ob.dr = ob.ret_dr("select isnull(max(sid),200)+1 from staff");
    if (ob.dr.Read())
    {
        c = Convert.ToInt32(ob.dr[0].ToString());
    }
    return c.ToString();
}
protected void Button1_Click(object sender, EventArgs e)
{
    if (TextBox1.Text == "" || TextBox2.Text == "" ||
DropDownList7.SelectedItem.ToString() == "Select")
    {
        Label11.Visible = true;
        Label11.ForeColor = System.Drawing.Color.Red;
        Label11.Text = "Please Enter all fields correctly!";
    }
    else
    {
        string aa = "NA";
        ob.dr = ob.ret_dr("select uname from Login where uname=" + TextBox1.Text +
""");
        if (!ob.dr.Read())
        {
            ob.exec("insert into staff values(" + TextBox1.Text + "," + TextBox2.Text +
"," + DropDownList7.SelectedItem.ToString() + "," + TextBox3.Text + "," +
TextBox8.Text + "," + TextBox9.Text + "," + TextBox10.Text + "," +
TextBox11.Text + "," + TextBox12.Text + ",0)");
            ob1.exec("insert into Login values(" + TextBox1.Text + "," + TextBox11.Text
+ "," + TextBox12.Text + ",2)");
            Label11.Visible = true;
            Label11.ForeColor = System.Drawing.Color.Green;
            Label11.Visible = true;

```

```
ScriptManager.RegisterStartupScript(this, this.GetType(), "message",
>alert('Details Added Successfully');", true);
Label11.ForeColor = System.Drawing.Color.Green;
Label11.Text = "Your request has been approved!";
TextBox1.Text = "";
TextBox2.Text = "";

TextBox3.Text = "";

TextBox1.Text = getid();

DropDownList7.SelectedIndex = 0;
}

}
}

protected void DropDownList7_SelectedIndexChanged(object sender, EventArgs e)
{

}
}
```

Admin Purchase History

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

public partial class AdminPurchaseHistory : System.Web.UI.Page
{

    DatabaseConnection ob = new DatabaseConnection();
    DatabaseConnection ob1 = new DatabaseConnection();
    public static string fpath = "", s = "", mode = "", str = "", userid;
    protected void Page_Load(object sender, EventArgs e)
    {
        if (!IsPostBack)
        {
            pub1();
        }
    }
    public DataSet pubdoc(string str)
    {
        DataSet ds = new DataSet();
```

```
try
{
    ds = ob.ret_ds(str);

    if (ds.Tables[0].Rows.Count > 0)
    {
        return ds;
    }
    else
    {
        return null;
    }
}
catch (Exception ex)
{
    return null;
}

}

public DataSet pubdoc1(string str)
{
    DataSet ds = new DataSet();
    try
    {
        ds = ob.ret_ds(str);

        if (ds.Tables[0].Rows.Count > 0)
        {
            return ds;
        }
        else
        {
            return null;
        }
    }
    catch (Exception ex)
    {
        return null;
    }
}

}

public void pub1()
{
    DataSet ds = new DataSet();
    try
    {
        string str = "select * from sales where status=1";
        ds = pubdoc1(str);
        if (ds != null)
```

```
        {
            GridView1.Visible = true;
            GridView1.DataSource = ds;
            GridView1.DataBind();
        }
        else
        {
            GridView1.Visible = false;
            //ScriptManager.RegisterStartupScript(this, this.GetType(), "message",
            "alert('Data Not Available');", true);

        }
    }
    catch (Exception ex)
    {
    }
}
}
```

Admin View Product Details

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

public partial class AdminViewProductDetails : System.Web.UI.Page
{
    DatabaseConnection ob = new DatabaseConnection();
    DatabaseConnection ob1 = new DatabaseConnection();
    protected void Page_Load(object sender, EventArgs e)
    {
        if (!IsPostBack)
        {
            pub();
        }
    }
    public DataSet pubdoc(string str)
    {
        DataSet ds = new DataSet();
        try
        {
            ds = ob.ret_ds(str);

            if (ds.Tables[0].Rows.Count > 0)
            {

```

```
        return ds;
    }
    else
    {
        return null;
    }
}
catch (Exception ex)
{
    return null;
}

}
public void pub()
{
    DataSet ds = new DataSet();
    try
    {
        string str = "select * from product where status=1 order by pcode asc";
        ds = pubdoc(str);
        if (ds != null)
        {

            gdImage.Visible = true;
            gdImage.DataSource = ds;
            gdImage.DataBind();
        }
        else
        {
            gdImage.Visible = false;
            ScriptManager.RegisterStartupScript(this, this.GetType(), "message",
"alert('Data Not Available');", true);

        }
    }
    catch (Exception ex)
    {
    }
}
}
```

Staff Update Stock

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
```

```
public partial class StaffAddStock : System.Web.UI.Page
{
    DatabaseConnection ob = new DatabaseConnection();
    DatabaseConnection ob1 = new DatabaseConnection();
    DataSet ds = new DataSet();
    public static int cstock, tstock;
    public static string suname = "";
    protected void Page_Load(object sender, EventArgs e)
    {
        if (!IsPostBack)
        {
            // Session["suname"] = "suresh";
            suname = Session["suname"].ToString();
            // Session["shopid"] = 104;
            DropDownList7.Items.Insert(0, new ListItem("SELECT", "0"));
            ob.dr = ob.ret_dr("select * from product where status=1");
            while (ob.dr.Read())
            {
                DropDownList7.Items.Add(ob.dr[0].ToString());
            }
            TextBox8.Text = DateTime.Now.ToShortDateString();
        }
    }
    public int CurrentPageIndex
    {
        get
        {
            if (ViewState["pg"] == null)
                return 0;
            else
                return Convert.ToInt16(ViewState["pg"]);
        }
        set
        {
            ViewState["pg"] = value;
        }
    }
    protected void DropDownList7_SelectedIndexChanged(object sender, EventArgs e)
    {
        CurrentPageIndex = 0;
        string catcode = "";
        PagedDataSource pgd = new PagedDataSource();
        try
        {
            ob.dr = ob.ret_dr("select * from product where pcode=" +
                DropDownList7.SelectedItem.ToString() + "");
            if (ob.dr.Read())
            {
                TextBox5.Text = ob.dr[3].ToString();
            }
        }
    }
}
```

```
        TextBox4.Text = ob.dr[1].ToString();
        TextBox9.Text = ob.dr[5].ToString();
        TextBox7.Text = ob.dr[6].ToString();

    }
    ob.dr = ob.ret_dr("select * from stock where pid=" +
DropDownList7.SelectedItem.ToString() + "");
    if (ob.dr.Read())
    {
        TextBox11.Text = ob.dr[0].ToString();
        TextBox10.Text = ob.dr[7].ToString();
        cstock = Convert.ToInt32(TextBox10.Text);

    }

    string str = "select pmimage,psimage1,psimage2,psimage3 from product where
pcode=" + DropDownList7.SelectedItem.ToString() + "";
    ds = ob1.ret_ds(str);
    pgd.DataSource = ds.Tables[0].DefaultView;
    //pgd.CurrentPageIndex = CurrentPageIndex;
    //pgd.AllowPaging = true;
    //pgd.PageSize = 4;
    DataList1.DataSource = pgd;
    DataList1.DataBind();
}
catch (Exception ex)
{
    Session["err"] = ex.ToString();
    Response.Redirect("error.aspx");
}
}

protected void Button1_Click(object sender, EventArgs e)
{
    tstock = cstock + Convert.ToInt32(TextBox10.Text);
    ob.exec("update stock set stock =" + tstock + ", uname='"+suname+"' where
stockid=" + TextBox11.Text + "");
    ob.exec("update product set status=1 where pcode=" +
DropDownList7.SelectedItem.ToString() + "");
    ScriptManager.RegisterStartupScript(this, this.GetType(), "message", "alert('Stock
Added Successfully');", true);

    TextBox4.Text = "";
    TextBox5.Text = "";
```

```
        TextBox7.Text = "";
        TextBox8.Text = "";
        TextBox9.Text = "";
        TextBox10.Text = "";

    }
}
```

Staff add production

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

public partial class StaffAddMilkProduction : System.Web.UI.Page
{
    DatabaseConnection ob = new DatabaseConnection();
    DatabaseConnection ob1 = new DatabaseConnection();
    public static string aa = "";
    public static string suname = "";
    protected void Page_Load(object sender, EventArgs e)
    {
        if (!IsPostBack)
        {
            // Session["suname"] = "suresh";
            suname = Session["suname"].ToString();
        }

        TextBox1.Text = getid();
        TextBox2.Text = suname;
        TextBox5.Text = DateTime.Now.ToShortDateString();
    }
    public string getid()
    {
        int c = 0;
        ob.dr = ob.ret_dr("select isnull(max(mid),700)+1 from milk");
        if (ob.dr.Read())
        {
            c = Convert.ToInt32(ob.dr[0].ToString());
        }
        return c.ToString();
    }
    protected void Button1_Click(object sender, EventArgs e)
    {
        if (TextBox1.Text == "" || TextBox2.Text == "")
        {
            Label11.Visible = true;
        }
    }
}
```

```

        Label11.ForeColor = System.Drawing.Color.Red;
        Label11.Text = "Please Enter all fields correctly!";
    }
    else
    {
        string aa = "NA";

        //string formattedDate = TextBox8.Text.ToString("dd-MM-yyyy",null);
        ob.exec("insert into milk values(" + TextBox1.Text + "," + TextBox2.Text + "," +
+ TextBox4.Text + "," + TextBox5.Text + ",0)");
        Label11.Visible = true;
        ScriptManager.RegisterStartupScript(this, this.GetType(), "message",
"alert('Details Added Successfully');", true);
        Label11.ForeColor = System.Drawing.Color.Green;
        Label11.Text = "Your request has been approved!";
        TextBox1.Text = "";
        TextBox2.Text = "";

        TextBox1.Text = getid();

    }
}
}

```

User Purchase Product

```

using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

public partial class UserProductList : System.Web.UI.Page
{
    DatabaseConnection ob = new DatabaseConnection();
    DatabaseConnection ob1 = new DatabaseConnection();
    DatabaseConnection ob2 = new DatabaseConnection();
    DataSet ds = new DataSet();
    int mid2, cartid;
    public static string uname, mimage, userid, catcode;
    string kfeature, accessories, spec = "";
    public static double offer = 0.0;
    public static string imgpath = "";
    protected void Page_Load(object sender, EventArgs e)
    {
        // Session["uname"] = "rakesh";
    }
}

```

```
        uname = Session["uname"].ToString();
        string mid1 = Request.QueryString[0];
        mid2 = Convert.ToInt32(mid1);

        if (!IsPostBack)
        {
            TextBox13.Text = "1".ToString();
            // BindData();
            GetFeatures();
            //GetFeatures1();
        }
    }
    public int CurrentPageIndex
    {
        get
        {
            if (ViewState["pg"] == null)
                return 0;
            else
                return Convert.ToInt16(ViewState["pg"]);
        }
        set
        {
            ViewState["pg"] = value;
        }
    }
    int pg = 0;
    protected void BindData()
    {
        CurrentPageIndex = 0;
        ds.Tables.Clear();
        PagedDataSource pgd = new PagedDataSource();
        string cmdstr = "Select * from product where pcode=" + mid2 + "";
        // string str = "select * from design";
        ds = ob1.ret_ds(cmdstr);
        pgd.DataSource = ds.Tables[0].DefaultView;
        pgd.CurrentPageIndex = CurrentPageIndex;
        pgd.AllowPaging = true;
        pgd.PageSize = 9;
        //LinkButton1.Enabled = !(pgd.IsLastPage);
        //LinkButton2.Enabled = !(pgd.IsFirstPage);

        // DataList1.DataSource = pgd;
        // DataList1.DataBind();
    }
    protected void GetFeatures()
    {
        ob.dr = ob.ret_dr("select * from product where pcode=" + mid2 + "");
    }
}
```

```
if (ob.dr.Read())
{
    Image1.ImageUrl = ob.dr[8].ToString();
    Image2.ImageUrl = ob.dr[9].ToString();
    Image3.ImageUrl = ob.dr[10].ToString();
    Image4.ImageUrl = ob.dr[11].ToString();
    Image5.ImageUrl = ob.dr[8].ToString();
    Image6.ImageUrl = ob.dr[9].ToString();
    Image7.ImageUrl = ob.dr[10].ToString();
    Image8.ImageUrl = ob.dr[11].ToString();
    Label1.Text = ob.dr[1].ToString();
    Label2.Text = ob.dr[6].ToString();
    Label3.Text = ob.dr[3].ToString();
    kfeature = ob.dr[4].ToString();
    //accessories = ob.dr[15].ToString();
    //spec = ob.dr[16].ToString();
    Label4.Text = ob.dr[2].ToString();

    // Label6.Text = ob.dr[4].ToString();

    imgpath = ob.dr[8].ToString();
    //catcode= ob.dr[1].ToString();
    // offer = Convert.ToDouble(ob.dr[10].ToString());
}
ListBox1.Items.Add(kfeature);

}

public string getid()
{
    int c = 0;
    ob.dr = ob.ret_dr("select isnull(max(cartid),700)+1 from cart");
    if (ob.dr.Read())
    {
        c = Convert.ToInt32(ob.dr[0].ToString());
    }
    return c.ToString();
}

public string getrate()
{
    int c1 = 0;
    ob.dr = ob.ret_dr("select rating from product where pcode=" + mid2 + "");
    if (ob.dr.Read())
    {
        c1 = Convert.ToInt32(ob.dr[0].ToString());
    }
    c1++;
    return c1.ToString();
}
```

```

    }

    protected void Button1_Click(object sender, EventArgs e)
    {
        string cart = getid();
        int stockno = 0;
        int salesno = 0;

        // double op1 = offer / 100;
        // double oprice = Convert.ToDouble(Label2.Text) * op1;
        // double dprice = (Convert.ToDouble(Label2.Text) *
        Convert.ToInt32(TextBox13.Text)) - (oprice * Convert.ToInt32(TextBox13.Text));
        // string iname = lblCity.Text + ":" + lblCName.Text + ":" + lblCName.Text + ":" +
        Label6.Text;
        //Label4.Text = op1.ToString();
        //Label5.Text = dprice.ToString();
        double dprice = Convert.ToDouble(Label2.Text);
        int nitmes = Convert.ToInt32(TextBox13.Text);
        double tamount = nitmes * dprice;
        cartid = Convert.ToInt32(cart);
        ob2.dr = ob2.ret_dr("select stock, sales from stock where pid=" + mid2 + "");
        if (ob2.dr.Read())
        {
            stockno = Convert.ToInt32(ob2.dr[0].ToString());
            salesno = Convert.ToInt32(ob2.dr[1].ToString());
        }
        if (stockno >= salesno)
        {
            ob.exec("insert into cart values(" + cartid + "," + mid2 + "," + Label1.Text + ",
            ""+imgpath+"" + "" + uname + "" + Label2.Text + "" + TextBox13.Text +
            ",0,"+tamount+",0)");
            //string rating = getrate();
            //ob1.exec("update product set rating=" + rating + " where pcode=" + mid2 + "");
            Response.Redirect("UserCheckOut.aspx");
        }
        else
        {
            Response.Write("<script LANGUAGE='JavaScript' >alert('Product Out of
            Stock')</script>");
        }
    }
}

```

User View Cart

```

using System;
using System.Collections;
using System.Collections.Generic;

```

```
using System.Data;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

public partial class UserCheckOut : System.Web.UI.Page
{
    DatabaseConnection ob = new DatabaseConnection();
    DatabaseConnection ob1 = new DatabaseConnection();
    DataSet ds = new DataSet();
    public static int count = 1;
    public static string uname = "";
    public static string dprice,tamt1, tquantity = "";
    public static int scount = 0;
    public static double tamt = 0.0;
    public int CurrentPageIndex
    {
        get
        {
            if (ViewState["pg"] == null)
                return 0;
            else
                return Convert.ToInt16(ViewState["pg"]);
        }
        set
        {
            ViewState["pg"] = value;
        }
    }
    int pg = 0;
    protected void BindData()
    {
        CurrentPageIndex = 0;
        ds.Tables.Clear();
        PagedDataSource pgd = new PagedDataSource();
        string cmdstr = "Select * from cart where uname='" + uname + "' and status=0";
        // string str = "select * from design";
        ds = ob1.ret_ds(cmdstr);
        pgd.DataSource = ds.Tables[0].DefaultView;
        pgd.CurrentPageIndex = CurrentPageIndex;
        pgd.AllowPaging = true;
        pgd.PageSize = 12;
        //LinkButton1.Enabled = !(pgd.IsLastPage);
        //LinkButton2.Enabled = !(pgd.IsFirstPage);

        DataList1.DataSource = pgd;
        DataList1.DataBind();
    }
    protected void Page_Load(object sender, EventArgs e)
```

```
{
    // Session["uname"] = "rakesh";
    uname = Session["uname"].ToString();
    if (!IsPostBack)
    {
        BindData();
        calculatetotal();
        shipping();
    }
}

public void shipping()
{
    ob.dr = ob.ret_dr("select * from registration where username='" + uname + "'and
status=0");
    if (ob.dr.Read())
    {
        TextBox13.Text = ob.dr[1].ToString();
        TextBox1.Text = ob.dr[2].ToString();
        TextBox2.Text = ob.dr[4].ToString();
        TextBox3.Text = ob.dr[5].ToString();
        TextBox4.Text = ob.dr[6].ToString();
        TextBox5.Text = ob.dr[12].ToString();
        TextBox6.Text = ob.dr[13].ToString();
        TextBox7.Text = ob.dr[14].ToString();

    }

}

public void calculatetotal()
{
    ob.dr = ob.ret_dr("select sum(CAST(price AS numeric)),sum(quantity),sum(tprice)
from cart where uname='" + uname + "'and status=0");
    // ob.dr = ob.ret_dr("select sum(price),sum(quantity) from cart where uname='" +
uname + "'and status=0");
    if (ob.dr.Read())
    {

        dprice = ob.dr[0].ToString();
        tquantity = ob.dr[1].ToString();
        tamt1 = ob.dr[2].ToString();
    }
    Label1.Text = tamt1.ToString();
}

public int getid()
{
    int c = 0;
    ob.dr = ob.ret_dr("select isnull(max(salesid),800)+1 from sales");
```

```

        if (ob.dr.Read())
        {
            c = Convert.ToInt32(ob.dr[0].ToString());
        }
        return c;
    }
    protected void Button1_Click(object sender, EventArgs e)
    {
        ArrayList al = new ArrayList();
        ArrayList al1 = new ArrayList();
        ob.dr = ob.ret_dr("select distinct pid,quantity from cart where uname='" + uname +
        "" and status=0");
        while (ob.dr.Read())
        {
            al.Add(ob.dr[0].ToString());
            al1.Add(ob.dr[1].ToString());
        }
        int uid = getid();
        string dd = DateTime.Now.ToShortDateString();
        DateTime pdate = DateTime.Now.AddDays(7);
        string pdate1 = pdate.ToShortDateString();
        ob.exec("insert into sales values(" + uid + "," + uname + "," + tqquantity + "," +
        dprice + "," + dd + "," + pdate1 + "','0')");
        ob1.exec("update cart set status=2 where status=0 and uname='" + uname + "'");
        int j = 0;
        foreach (string i in al)
        {
            ob.dr = ob.ret_dr("select sales from stock where pid=" + i + "");
            if (ob.dr.Read())
            {
                scout = Convert.ToInt32(ob.dr[0].ToString());

            }
            int scout1 = scout + Convert.ToInt32(al1[Convert.ToInt32(j)]);
            ob1.exec("update stock set sales=" + scout1 + " where pid=" + i + "");
            j++;
        }
        Response.Redirect("UserConfirmation.aspx");
    }

    protected void DataList1_SelectedIndexChanged(object sender, EventArgs e)
    {
        string c = DataList1.DataKeys[DataList1.SelectedIndex].ToString();
        ob1.exec("delete from cart where cartid=" + c + "");
        BindData();
        calculatetotal();
    }
}

```

User Milk Subscription

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

public partial class UserMilkSubscription : System.Web.UI.Page
{
    DatabaseConnection ob = new DatabaseConnection();
    DatabaseConnection ob1 = new DatabaseConnection();
    public static string uname = "";
    public static string pid, cartid = "";
    protected void Page_Load(object sender, EventArgs e)
    {
        if (!IsPostBack)
        {
            DropDownList1.Items.Insert(0, new ListItem("SELECT", "0"));
            DropDownList1.Items.Insert(1, new ListItem("1", "1"));
            DropDownList1.Items.Insert(2, new ListItem("2", "2"));
            DropDownList1.Items.Insert(3, new ListItem("3", "3"));
            DropDownList1.Items.Insert(4, new ListItem("4", "4"));
            DropDownList1.Items.Insert(5, new ListItem("5", "5"));
            DropDownList1.Items.Insert(6, new ListItem("6", "6"));
            //TextBox3.Text = Session["shopid"].ToString();

            //TextBox2.Text = Session["productname"].ToString();
            Session["uname"] = "rakesh";
            TextBox1.Text = Session["uname"].ToString();
            uname = Session["uname"].ToString();

        }
    }
    public string getid()
    {
        int c = 0;
        ob.dr = ob.ret_dr("select isnull(max(subid),600)+1 from subscription");
        if (ob.dr.Read())
        {
            c = Convert.ToInt32(ob.dr[0].ToString());
        }
        return c.ToString();
    }
    protected void DropDownList1_SelectedIndexChanged(object sender, EventArgs e)
    {
    }
}
```

```
protected void Button1_Click(object sender, EventArgs e)
{
    if (TextBox1.Text == "" || TextBox2.Text == "" ||
DropDownList1.SelectedItem.ToString() == "Select")
    {
        Label11.Visible = true;
        Label11.ForeColor = System.Drawing.Color.Red;
        Label11.Text = "Please Enter all fields correctly!";
    }
    else
    {
        string fid = getid();
        ob.exec("insert into subscription values(" + fid + "," + TextBox1.Text + "," +
DropDownList1.SelectedItem.ToString() + "," + TextBox2.Text + "," + TextBox8.Text
+ ",0)");

        Label11.Visible = true;
        Label11.ForeColor = System.Drawing.Color.Green;
        Label11.Text = "Your request has been approved!";
        TextBox1.Text = "";
        TextBox2.Text = "";

    }
}
```

CHAPTER 5

SYSTEM TESTING

System Testing is the stage of implementation, which is aimed at ensuring that the system works accurately and efficiently as expected before live operation commences. It certifies that the whole set of program hang together. System testing requires a test plan that consists of several keys, activities and steps to run program, string, system and user acceptance testing. The implementation of newly designed package is important in adopting a successful new system.

TESTING OBJECTIVES

- Testing is the process of correcting a program with intend of finding an error.
- A good test is one that has a high probability of finding a yet undiscovered error.
- A successful test is one that uncovers an undiscovered error.

There are different types of testing methods available:

5.1UNIT TESTING

In this testing we test each module individually and integrate the overall system. Unit testing focuses verification efforts on the smaller unit of software design in the module. This is also known as ‘module’ testing. The modules of the system are tested separately. The testing is carried out during programming stage itself. In this testing step each module is found to work satisfactory as regard to the expected output from the module. There are some validation checks for verifying the data input given by the user. It is very easy to find error and debug the system.

5.2INTEGRATION TESTING

Data can be lost across an interface; one module can have an adverse effect on the other sub functions when combined by May not produce the desired major functions. Integrated testing is the systematic testing for constructing the uncover errors within the interface. This testing was done with sample data. The need for integrated test is to find the overall system performance.

5.3BLACK BOX TESTING

This testing attempts to find errors in the following areas or categories: Incorrect or missing functions, interface errors, errors in data structures, external database access, performance errors and initialization and termination errors.

5.4 VALIDATION TESTING

At the culmination of Black Box testing, software is completely assembled as a package, interface errors have been uncovered and corrected and final series of software tests, validation tests begins. Validation testing can be defined in many ways but a simple definition is that validation succeeds when the software functions in a manner that can be reasonably accepted by the customer.

After validation test has been conducted one of the two possible conditions exists.

- The function or performance characteristics confirm to specification and are accepted.
- A deviation from specification is uncovered and a deficiency list is created.

5.5 OUTPUT TESTING

After performing the validation testing, the next step is output testing of the proposed system since no system could be useful if it doesn't produce the required data in the specific format. The output displayed or generated by the system under consideration is tested by, asking the user about the format displayed. The output format on the screen is found to be correct as the format was designed in the system according to the user needs. Hence the output testing doesn't result in any correction of the system.

5.6 USER ACCEPTANCE TESTING

User acceptance of the system is the key factor for the success of the system. The system under consideration is tested for user acceptance by constantly keeping in touch with prospective system at the time of developing and making change wherever required. This is done with regard to the following points:

- Output Screen design.
- Input Screen design.
- Menu driven system.

5.7 WHITE BOX TESTING

White box testing is a testing case design method that uses the control structure of the procedural design to derive the test cases. The entire independent path in a module is exercised at least once. All the logical decisions are exercised at least once. Executing all the loops at boundaries and within their operational bounds exercise internal data structure to ensure their validity.

In my project testing was conducted at every step. Initially each module was tested separately to check whether they gave the desired output for the given input. The forms used to enter data by user were validated and appropriate error messages were displayed if incorrect data was entered. Once the data was entered correctly, the processing was done and testing was done to check whether the correct output was obtained. Once the test cases were conducted successfully for each module, the modules were integrated together as a single system. After integration, the test cases were again applied to check whether the entire system as a whole produced the desired output. Once the integration testing was performed correctly, output testing was done and it did not result in any change or correction in the system. Black box testing and white box testing was also conducted successfully. All the loops, decisions, relations were executed at least once before giving it to the users for testing. In black box testing, it was checked whether the data in the proper format was stored in the database or not. Also, it was checked whether the interfaces were working properly or not. On successful completion of these tests, the system was then given to undergo user acceptance testing where the users entered test data to check whether the correct output was obtained. The users were satisfied with the output and thus the testing phase was completed successfully.

5.8 TEST DATA AND RESULTS

The primary goal of software implementation is the production of source code that is easy to read and understand. Clarification of source code helps in easier debugging, testing and modification. Source code clarification is enhanced by structural coding techniques, by good coding style, by appropriate supporting documents, by good internal comments and by the features provided in the modern programming language.

In our implementation phase, source code contains both global and formal variables. It contains predefined functions as well as the user defined functions. The result of the new system is compared with old system and supposes if the result is wrong the error must be debugged.

After the acceptance of the system by the user, the existing system should be replaced by this system. Any user handles this package very easily. It does not require any intensive training for the user. Procedures and functions involved in this system are very simple that anyone can understand and correspondingly act to the system with no difficulty.

CHAPTER 6

SYSTEM IMPLEMENTATION AND MAINTENANCE

The maintenance phase of software life cycle is the time period in which software performs useful work. We may define maintenance by describing for activities that are undertaken after a programs released for use. During the use of any large program, errors will occur and be reported to the developer. This process that includes the diagnosis and correction of one or more is called corrective maintenance. Thus problem correction involves modification and revalidation of software to corrective maintenance.

The second activity that contributes to a definition of maintenance occurs because of the rapid change that is encountered in every aspect of computing. An activity that modifies software to properly interface with a changing environment is adaptive maintenance. The third activity occurs when a software package is successful. As software is used recommendation for new capabilities, modifications to existing function and general enhancements are received from users. To satisfy request in the category, perfective maintenance is performed. The fourth and the last maintenance activity occurs when software is changed to improve future maintainability or reliability or to provide activities including correcting, coding and design, updating, documentation and test data and upgrading user support. Maintenance means restoring something to its original conditions, in contrast, enhancements means adding or modifying or execution.

CHAPTER 7

FUTURE ENHACEMENT

To further improve the functionality and efficiency of the E-Dairy Farm System, several future enhancements can be considered:

1. **AI-Powered Cattle Health Monitoring** – Integrating AI and IoT-based sensors to monitor cattle health in real-time, detecting diseases early and optimizing veterinary care.
2. **Automated Milk Quality Testing** – Implementing automated systems to test milk quality based on parameters like fat content, bacterial count, and freshness to ensure high standards.
3. **Mobile Application** – Developing a dedicated mobile app for farmers, staff, and customers to manage farm operations and place orders conveniently from anywhere.
4. **Blockchain for Transparency** – Utilizing blockchain technology to ensure transparency in transactions, milk sourcing, and product authenticity for better trust among consumers.
5. **Predictive Analytics** – Using data analytics to forecast milk production trends, optimize inventory, and improve overall farm productivity.
6. **Subscription-Based Milk Delivery** – Introducing a subscription model where customers can receive regular deliveries of dairy products based on their preferences.
7. **Integration with Smart Payment Systems** – Supporting multiple payment gateways, including digital wallets, UPI, and cryptocurrency for secure and flexible transactions.
8. **Automated Supply Chain Management** – Enhancing logistics with real-time tracking of product deliveries and automated restocking of inventory.

CHAPTER 8

CONCLUSION

The E-Dairy Farm System is a comprehensive and efficient solution designed to streamline dairy farm operations by automating essential tasks such as cattle management, milk production tracking, and online sales. By integrating advanced features like automated transactions, inventory management, and an intuitive interface, the system significantly reduces manual workload and enhances accuracy.

With distinct roles for administrators, staff, and the public, the platform ensures smooth management and accessibility for all users. Administrators have full control over farm operations, staff efficiently handle day-to-day activities, and customers can conveniently purchase dairy products online.

Overall, the E-Dairy Farm System modernizes traditional dairy farming, increasing productivity, improving record-keeping, and making dairy products more accessible to consumers. It serves as a complete digital solution that enhances efficiency and profitability for dairy farm businesses.

CHAPTER 9

APPENDIX

Home Page



User Registration

DiaryFarm

HomeAboutUser RegistrationLoginContact

25
Years Experience

Create your account

Username:

Rakesh

Name:

Rakesh

Email ID:

rakesh@gmail.com

Password:

••••••••

Confirm Password:

••••••••

Contact Number:

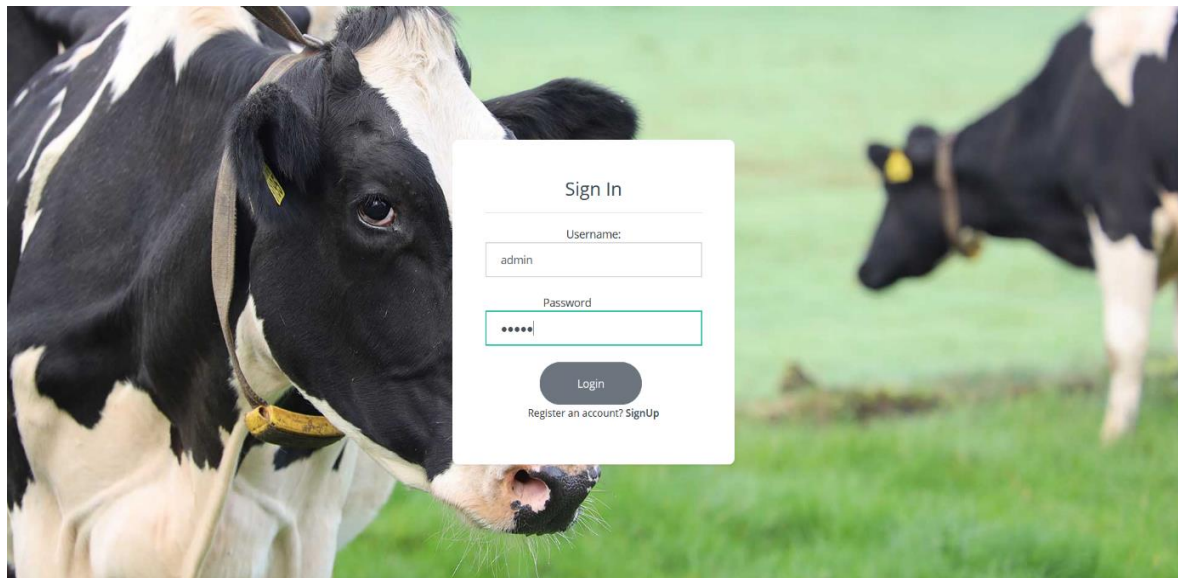
9878767676

Country:

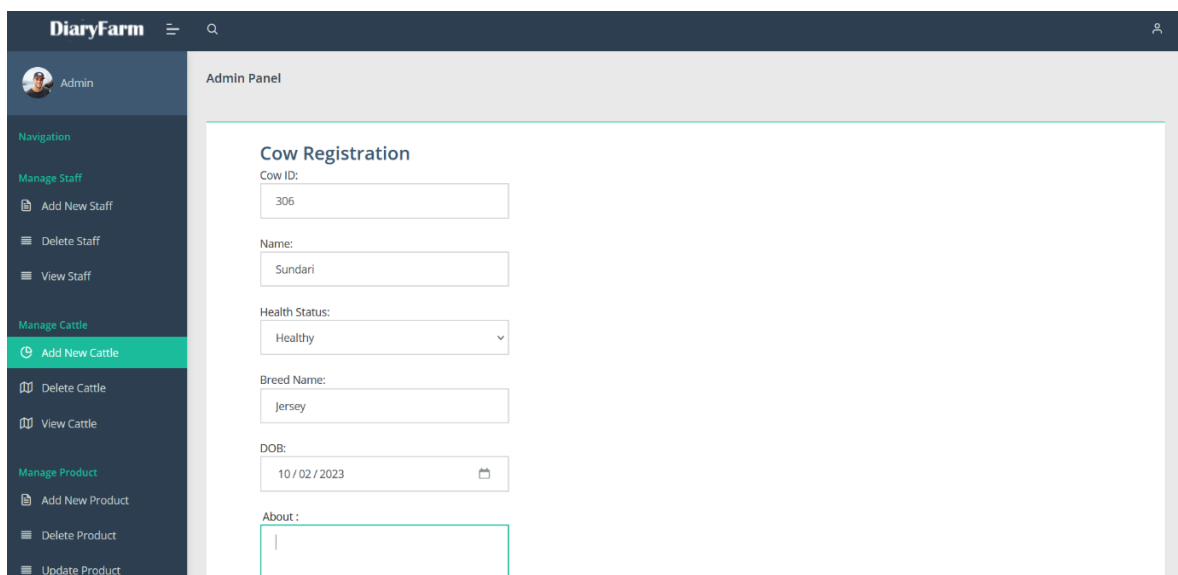
INDIA

AddressLine1:

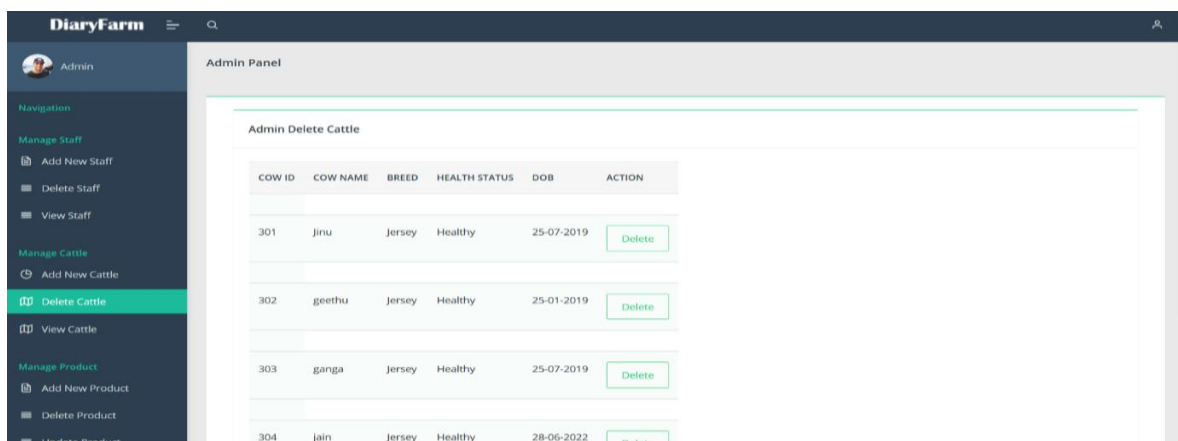
Login



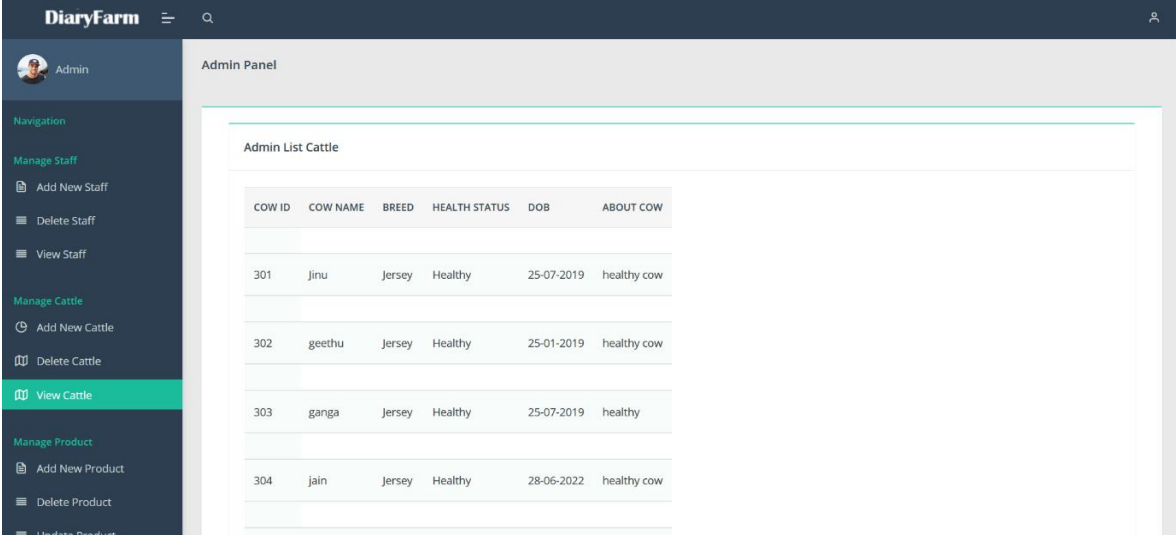
Admin Add Cattle



Admin Delete Cattle



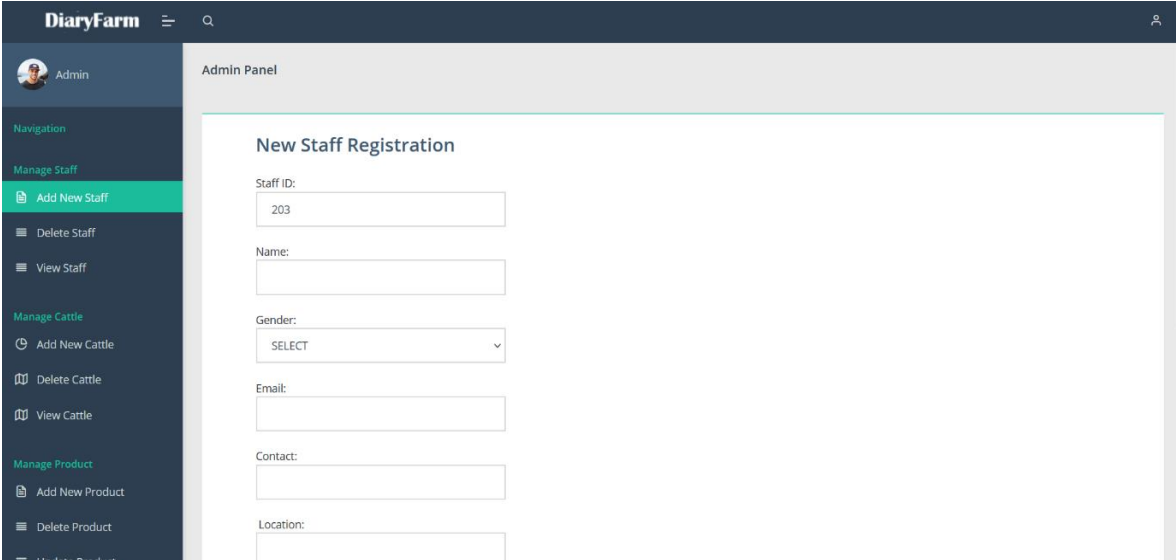
Admin List Cattle



Admin List Cattle

COW ID	COW NAME	BREED	HEALTH STATUS	DOB	ABOUT COW
301	Jinu	Jersey	Healthy	25-07-2019	healthy cow
302	geethu	Jersey	Healthy	25-01-2019	healthy cow
303	ganga	Jersey	Healthy	25-07-2019	healthy
304	jain	Jersey	Healthy	28-06-2022	healthy cow

New staff registration



New Staff Registration

Staff ID:

Name:

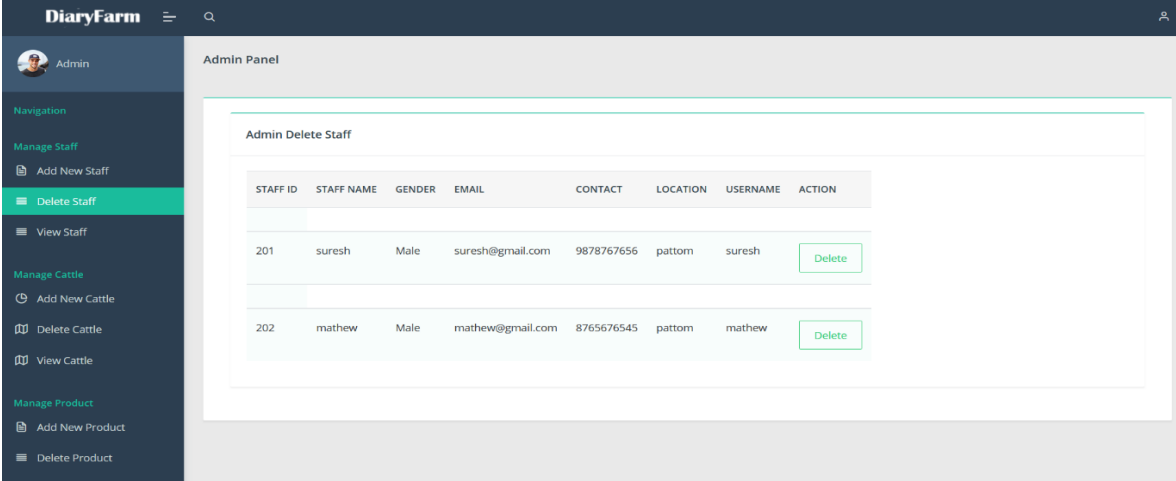
Gender:

Email:

Contact:

Location:

Admin Delete Staff



Admin Delete Staff

STAFF ID	STAFF NAME	GENDER	EMAIL	CONTACT	LOCATION	USERNAME	ACTION
201	suresh	Male	suresh@gmail.com	9878767656	pattom	suresh	Delete
202	mathew	Male	mathew@gmail.com	8765676545	pattom	mathew	Delete

Admin View Staff

Admin Panel

Admin View Staff

STAFF ID	STAFF NAME	GENDER	EMAIL	CONTACT	LOCATION	USERNAME
201	suresh	Male	suresh@gmail.com	9878767656	pattom	suresh
202	mathew	Male	mathew@gmail.com	8765676545	pattom	mathew

Shipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share

Admin View Product Details

Admin Panel

View Product Details

PID	Title	Product Name	Quantity	Type	Price	MainImage	SubImage	SubImage	SubImage	Actions
501	Farm Fresh Cow Milk	Cow milk	500 ml	Milk	25					Delete
502	Natural Cow Ghee	Cow Ghee	500 ml	Ghee	345					Delete
503	Pravarsha Thick Curd 5kg	Thick Curd	5 kg	Curd	500					Delete

Admin Update Product

Admin Panel

Update Product

Product Code: 502

Product Name: Cow Ghee

Quantity: 500 ml

Product Title: Natural Cow Ghee

Price (Rs.): 345

Product Type: Ghee

Admin View Purchase History

The screenshot shows the Admin Panel interface. On the left is a navigation sidebar with sections: Navigation, Manage Staff, Manage Cattle, and Manage Product. The main content area is titled 'Admin Panel' and contains a 'Purchase History' section. Below this title is a table with the following data:

Item ID	Username	Total Amount	Purchase Date	Delivery Date
801	rakesh	1190	03/03/2025	10/03/2025

Staff Update Stock

The screenshot shows the Staff Panel interface. On the left is a navigation sidebar with sections: Navigation, Manage Product, Manage Milk, and Manage Cattle. The main content area is titled 'Admin Panel' and contains a form for updating stock. The form fields are:

- Product ID: 502 (dropdown)
- Stock ID: 902 (text)
- Stock Update: 20 (text)
- Submit button
- Product Name: Cow Ghee (text)
- Product Title: Natural Cow Ghee (text)
- Product Price Rs: 345 (text)
- Stock Date: 02/04/2025 (text)
- Product Type: Ghee (text)

Below the form, there are four images of the product: a large jar of Pravarsha Cow Ghee, a smaller jar, a nutritional facts label, and another jar.

Staff View Product

The screenshot shows the Staff Panel interface. On the left is a navigation sidebar with sections: Navigation, Manage Product, Manage Milk, and Manage Cattle. The main content area is titled 'Admin Panel' and contains a 'View Product Details' section. Below this title is a table with the following data:

PID	Title	Product Name	Quantity	Type	Price	MainImage	SubImage	SubImage	SubImage
501	Farm Fresh Cow Milk	Cow milk	500 ml	Milk	25				
502	Natural Cow Ghee	Cow Ghee	500 ml	Ghee	345				
503	Pravarsha Thick Curd 5kg	Thick Curd	5 kg	Curd	500				

Staff Update Milk Production

DiaryFarm

Staff

Navigation

Manage Product

Add Stock

View Product

Manage Milk

Add Milk Production

View Milk Production

Graphical Analysis

Manage Cattle

Add Health Details

View Health Details

View Cattle

Admin Panel

Dilay Update Milk Production

Milk ID:

704

Staff Name:

suresh

Daily Milk Production:

Date:

02/04/2025

Submit

Staff View Milk Production

Add Stock

View Product

Manage Milk

Add Milk Production

View Milk Production

Graphical Analysis

Manage Cattle

Add Health Details

View Health Details

View Cattle

Manage Sales

Purchase Product

Purchase History

Manage Complaints

Send Complaints

02 / 03 / 2025

Ending Date:

15 / 03 / 2025

Search

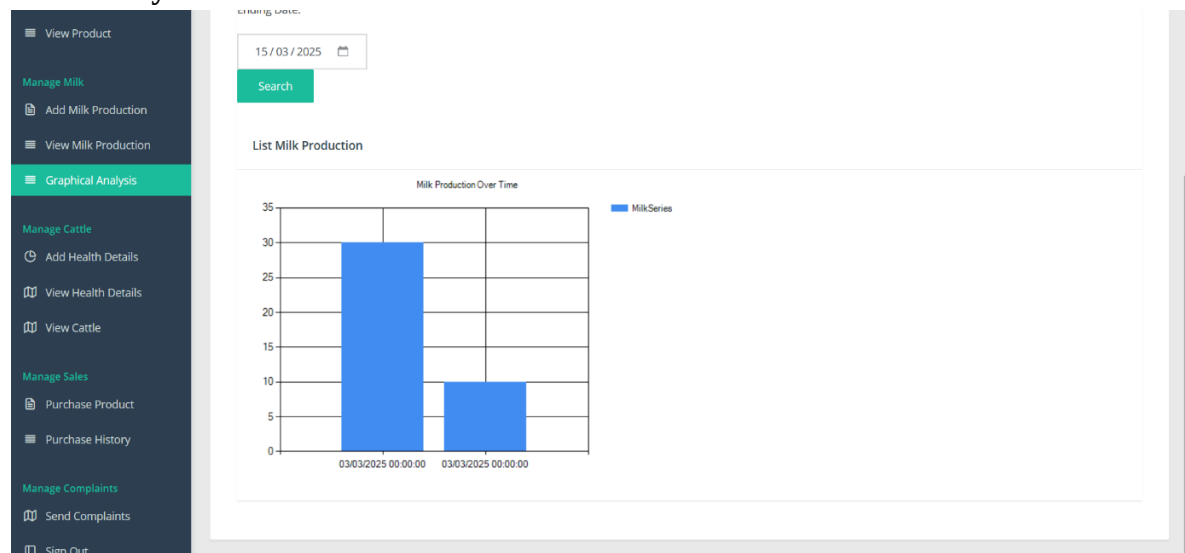
List Milk Production

MILK ID	STAFF USERNAME	PRODUCTION (IN LITRE)	DATE
701	suresh	30	03-03-2025
702	suresh	10	03-03-2025

Total Production of Milk in Litres:

40

Staff Analysis Data



Staff View Health Details

DiaryFarm

≡

Q

Staff

Navigation

Manage Product

Add Stock

View Product

Manage Milk

Add Milk Production

View Milk Production

Graphical Analysis

Manage Cattle

Add Health Details

View Health Details

View Cattle

Admin Panel

Cow ID:

SELECT

Cow Health Details

COW ID	COW NAME	WEIGHT	DOCTOR NAME	CHECKED DATE	VACCINATION DATE	HEALTH STATUS
301	Jinu	75 kg	george	03-03-2025	21-03-2025	good condition
303	ganga	80 kg	rakesh	03-03-2025	13-03-2025	good condition
302	geethu	80 kg	george	03-03-2025	21-03-2025	good condition

Staff View Purchase History

DiaryFarm

≡

Q

Staff

Navigation

Manage Product

Add Stock

View Product

Manage Milk

Add Milk Production

View Milk Production

Graphical Analysis

Manage Cattle

Add Health Details

View Health Details

View Cattle

Admin Panel

View User Product Details

Design	Product Name	ID	Price	Username	Q
	Cow Ghee	502	345	rakesh	2
	Thick Curd	503	500	rakesh	3
	Cow Ghee	502	345	rakesh	1

Staff Send Complaints

DiaryFarm

≡

Q

Staff

Navigation

Manage Product

Add Stock

View Product

Manage Milk

Add Milk Production

View Milk Production

Graphical Analysis

Manage Cattle

Add Health Details

View Health Details

View Cattle

Admin Panel

Complaint Registration

Subject:

Complaints:

Register

ID	Subject	Mail	Send Date	Reply	Reply Date
602	test	test.mrddddd	03/03/2025	will check	03/03/2025
603	dfdf	dfdf	03/03/2025	will check	03/03/2025
1					

User View Product

DiaryFarm

9847207876 info@diaryfarm.com MG Road, Trivandrum

[Home](#)
[Product](#)
[Category](#)
[Cart](#)
[History](#)
[Complaint](#)
[FeedBack](#)
[View FeedBack](#)
[Subscription](#)
[SignOut](#)

PRODUCT CATEGORY

Curd

Ghee

Milk

PRODUCT NAME

Cow Ghee

Cow milk

Thick Curd

Cow milk

PRODUCT

FARM FRESH COW MILK

Rs.25

★★★★

Cow Ghee

PRODUCT

NATURAL COW GHEE

Rs.345

★★★★

Thick Curd

PRODUCT

PRAVARSHA THICK CURD 5KG

Rs.500

★★★★

+

-

User Purchase Product

DiaryFarm

9847207876 info@diaryfarm.com MG Road, Trivandrum

[Home](#)
[Product](#)
[Category](#)
[Cart](#)
[History](#)
[Complaint](#)
[FeedBack](#)
[View FeedBack](#)
[Subscription](#)
[SignOut](#)

PRODUCT

FARM FRESH COW MILK

Rs.25

★★★★

COW MILK

★★★★

Rs.25 IN STOCK

Farm Fresh Cow Milk.

QUANTITY : 500 ML

QTY

ADD TO CART

User View Cart

BILLING ADDRESS

rakesh

rakesh@gmail.com

9878767876

ss nagarpalayamtrivandrum

MALE

KERALA

trivandrum

695003

SHIPPING ADDRESS

☐ Ship to a different address?

YOUR ORDER

PRODUCT	PRICE	TOTAL AMOUNT	REMOVE
1 x Cow milk	Rs. 25	Rs. 25	
1 x Cow milk	Rs. 25	Rs. 25	
Shipping			FREE
TOTAL			Rs. 50

☐ Direct Bank Transfer
 ☐ Cheque Payment
 ☐ Paypal System

☐ I've read and accept the terms & conditions

PLACE ORDER

User View Purchase History

DiaryFarm

9847207876 info@diaryfarm.com MG Road, Trivandrum

[Home](#) [Product](#) [Category](#) [Cart](#) [History](#) [Complaint](#) [FeedBack](#) [View FeedBack](#) [Subscription](#) [SignOut](#)

Purchase History

Pending History

Item ID	User Name	Total Amount	Purchase Date	Expected Delivery Date	Select
801	rakesh	1190	03/03/2025	10/03/2025	Item Received
802	rakesh	25	03/03/2025	10/03/2025	Item Received

Delevered History

Item ID	Username	Total Amount	Purchase Date	Delivery Date
801	rakesh	1190	03/03/2025	10/03/2025

Copyright ©2025 All rights reserved | This template is made with ♥ by [UI-Kit](#)

User send Feedback

DiaryFarm

9847207876 info@diaryfarm.com MG Road, Trivandrum

[Home](#) [Product](#) [Category](#) [Cart](#) [History](#) [Complaint](#) [FeedBack](#) [View FeedBack](#) [Subscription](#) [SignOut](#)

User FeedBack Form

Username:

rakesh

Purchase ID:

705

Product Name:

Cow milk

Rating:

2

FeedBack :

good

Submit

Cancel

Copyright ©2025 All rights reserved | This template is made with ♥ by [UI-Kit](#)

User update Milk Subscription

DiaryFarm

9847207876 info@diaryfarm.com MG Road, Trivandrum

[Home](#) [Product](#) [Category](#) [Cart](#) [History](#) [Complaint](#) [FeedBack](#) [View FeedBack](#) [Subscription](#) [SignOut](#)

User Add Milk Subscription

Username:

rakesh

Milk (Litre):

3

Starting Date:

18 / 04 / 2025

Ending Date:

23 / 05 / 2025

Submit

Cancel

Copyright ©2025 All rights reserved | This template is made with ♥ by [UI-Kit](#)

CHAPTER 9

BIBLIOGRAPHY

Books

- Beginning Visual C# -Watson,White, Wrox Publications.
- C# Programming Bible-Jeff Ferguson, Meeta Gupta.
- Pro ASP.Net 4.0 in C# -Mac Donald and Szpuszta.
- Software Engineering-Roger S Pressman.
- System Analysis and Design- James A. Senn.

Internet sites

- www.msdn.microsoft.com.
- http://en.wikipedia.org/wiki/Internet_Information_Services.
- http://en.wikipedia.org/wiki/C_Sharp.
- http://en.wikipedia.org/wiki/Windows_Server_2003.
- http://en.wikipedia.org/wiki/Microsoft_Visual_Studio

CHAPTER 10

GANTT CHART

Gantt chart shows the time relationship between ‘events’ of the production program has regarded as revolutionary in management. Gantt chart recognize the total program goals and it should be regarded as a series of inter - related supporting plan (or events), that people can comprehend and follow.

The following figure is the Gantt Chart of our project. The plan explains the task verses the time they will take to complete.

Sl. No.	Task Name	Duration	Start Design	End Design	January				February				March				April			
					W	W	W	W	W	W	W	W	W	W	W	W	W	W	W	W
					1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	Analysis	7	03.01.2025	10.01.2025																
2	System Design	20	24.01.2025	14.02.2025																
3	Code Design	23	17.02.2025	12.03.2025																
4	Testing	16	15.03.2025	25.03.2025																
5	Documentation	14	28.03.2025	02.04.2025																